

UNIVERSIDADE FEDERAL DE JUIZ DE FORA  
FACULDADE DE ENGENHARIA  
GRADUAÇÃO EM ENGENHARIA ELÉTRICA

Pedro Henrique Assis Silva

Técnicas de *Machine Learning* Para Predição de Doenças Cardiovasculares

Juiz de Fora

2023

Pedro Henrique Assis Silva

Técnicas de *Machine Learning* Para Predição de Doenças Cardiovasculares

Trabalho de conclusão de curso apresentado ao programa de graduação em engenharia elétrica da Universidade Federal de Juiz de Fora como requisito parcial à obtenção do título de engenheiro eletricista.

Orientadora: Prof<sup>ca</sup>. Dr<sup>a</sup> Ana Sophia Cavalcanti Alves Vilas Boas

Juiz de Fora

2023

Ficha catalográfica elaborada através do Modelo Latex do CDC da UFJF  
com os dados fornecidos pelo(a) autor(a)

Silva, Pedro Henrique Assis.

Técnicas de *Machine Learning* Para Predição de Doenças Cardiovasculares / Pedro Henrique Assis Silva. – 2023.

58 f. : il.

Orientadora: Ana Sophia Cavalcanti Alves Vilas Boas

Trabalho de Conclusão de Curso – Universidade Federal de Juiz de Fora, Faculdade de Engenharia. Graduação em Engenharia Elétrica, 2023.

1. Banco de Dados. 2. Doenças Cardiovasculares. 3. *Machine Learning*.  
I. Boas, A. S. C. A. V.

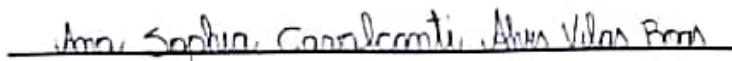
Pedro Henrique Assis Silva

**Técnicas de *Machine Learning* Para Predição de Doenças Cardiovasculares**

Trabalho de conclusão de curso apresentado  
ao programa de graduação em engenharia  
elétrica da Universidade Federal de Juiz de  
Fora como requisito parcial à obtenção do  
título de engenheiro eletricitista.

Aprovado em 23 de junho de 2023

**BANCA EXAMINADORA**

  
Prof<sup>a</sup>. Dr<sup>a</sup>. Ana Sophia Cavalcanti Alves Vilas Boas  
Orientadora  
Universidade Federal de Juiz de Fora

  
Prof. Dr. Daniel de Almeida Fernandes  
Universidade Federal de Juiz de Fora

Dedico o presente trabalho a comunidade acadêmica.

## AGRADECIMENTOS

Gostaria de expressar minha profunda gratidão a Deus, por ter me dado força, saúde e sabedoria, permitindo-me superar os desafios ao longo desta longa jornada.

Agradeço aos meus amados pais, Rita Elena e José Pedro. Sou imensamente grato por todo o amor incondicional e carinho constante que vocês me proporcionaram ao longo desta jornada.

Sou grato aos meus familiares e amigos pelo apoio incondicional, encorajamento e compreensão durante toda a jornada acadêmica. Em especial, expresso minha gratidão aos meus tios, Rosana e Rodrigo, que generosamente abriram a porta de sua casa e me receberam com muito carinho.

Agradeço à minha orientadora, Ana Sophia, pela orientação e suporte ao longo de todo o processo. Também agradeço aos professores e corpo docente do curso de Engenharia Elétrica, por compartilharem seu conhecimento e proporcionarem uma sólida base acadêmica.

Agradeço aos meus colegas de turma e demais colegas de curso, pela troca de ideias, discussões enriquecedoras e apoio mútuo ao longo desses anos.

Por fim, agradeço a todos os autores, pesquisadores e desenvolvedores cujos trabalhos foram fundamentais para a realização deste estudo. Suas contribuições acadêmicas e técnicas foram essenciais para o desenvolvimento deste trabalho.

Expresso minha profunda gratidão a todos mencionados e a todos aqueles que, de alguma forma, contribuíram para o sucesso deste trabalho.

Obrigado a todos!

*“Tudo o que a mente humana pode conceber, ela pode conquistar.”*

*(Napoleon Hill)*

## RESUMO

As doenças cardiovasculares representam um problema de saúde pública global de grande magnitude, sendo responsáveis por uma das principais causas de mortalidade no mundo. A prevenção e o tratamento efetivo dessas doenças dependem de uma identificação precoce do risco em indivíduos específicos. No entanto, essa identificação é baseada em dados clínicos e médicos que frequentemente são incompletos ou imprecisos, o que dificulta a análise manual desses dados e aumenta a possibilidade de erros humanos. Para superar esse obstáculo, técnicas de *machine learning* (aprendizado de máquina) têm sido amplamente utilizadas na área médica para prever o risco de doenças cardiovasculares. Essas técnicas são capazes de processar grandes volumes de dados de forma rápida e eficiente, permitindo uma identificação precoce de indivíduos com maior probabilidade de desenvolver essas doenças. Além disso, a disponibilidade de diversas bases de dados públicas possibilita que os pesquisadores treinem modelos de aprendizado de máquina com um grande número de dados, tornando esses modelos ainda mais precisos e confiáveis. Dentre as técnicas de *machine learning* aplicadas ao contexto das doenças cardiovasculares, destacam-se a regressão logística, o *k-nearest neighbors* (*k-NN*) e o *random forest*. A regressão logística é um modelo estatístico que permite estimar a probabilidade de ocorrência de um evento, como a presença de uma doença cardiovascular, com base em variáveis preditoras. O *k-NN* é um algoritmo de classificação que utiliza a similaridade entre amostras para atribuir rótulos a novos dados. Já o *random forest* é um método de aprendizado baseado em árvores de decisão, que combina várias árvores individuais para obter uma previsão mais robusta. A avaliação desses modelos de *machine learning* é essencial para determinar sua eficácia na previsão do risco de doenças cardiovasculares. Métricas como acurácia, precisão, *recall* e *F1-score* são comumente utilizadas para avaliar o desempenho desses modelos. Com o uso de técnicas de *machine learning*, como a regressão logística, o *k-NN* e o *random forest*, é possível obter previsões mais precisas e confiáveis sobre o risco de doenças cardiovasculares. Essas abordagens oferecem uma alternativa promissora para auxiliar os profissionais de saúde na tomada de decisões clínicas, contribuindo para uma identificação precoce e um tratamento mais efetivo dessas doenças.

Palavras-chave: Banco de Dados. Doenças Cardiovasculares. *Machine Learning*.



## ABSTRACT

Cardiovascular diseases represent a major global public health problem, being one of the leading causes of mortality worldwide. The prevention and effective treatment of these diseases depend on early identification of risk in specific individuals. However, this identification is based on clinical and medical data that are often incomplete or inaccurate, making manual analysis challenging and increasing the possibility of human errors. To overcome this obstacle, *machine learning* techniques have been widely used in the medical field to predict the risk of cardiovascular diseases. These techniques are capable of processing large volumes of data quickly and efficiently, enabling early identification of individuals with a higher likelihood of developing these diseases. Moreover, the availability of diverse public databases allows researchers to train *machine learning* models with a large amount of data, making these models even more accurate and reliable. Among the *machine learning* techniques applied to the context of cardiovascular diseases, logistic regression, k-NN (k-Nearest Neighbors), and Random forest are notable. Logistic regression is a statistical model that estimates the probability of an event, such as the presence of a cardiovascular disease, based on predictor variables. k-NN is a classification algorithm that uses the similarity between samples to assign labels to new data. Random forest, on the other hand, is a learning method based on decision trees that combines multiple individual trees to obtain a more robust prediction. The evaluation of these *machine learning* models is essential to determine their effectiveness in predicting the risk of cardiovascular diseases. Metrics such as accuracy, precision, recall, and F1-score are commonly used to assess the performance of these models. By using *machine learning* techniques such as logistic regression, k-NN, and random forest, it is possible to obtain more accurate and reliable predictions about the risk of cardiovascular diseases. These approaches offer a promising alternative to assist healthcare professionals in clinical decision-making, contributing to early identification and more effective treatment of these diseases.

Keywords: Cardiovascular. Diseases. Database. *Machine Learning*.

## SUMÁRIO

|              |                                                                    |           |
|--------------|--------------------------------------------------------------------|-----------|
| <b>1</b>     | <b>INTRODUÇÃO . . . . .</b>                                        | <b>11</b> |
| 1.1          | CONTEXTUALIZAÇÃO E MOTIVAÇÃO . . . . .                             | 11        |
| 1.2          | OBJETIVOS . . . . .                                                | 12        |
| 1.3          | ESTRUTURA DO TRABALHO . . . . .                                    | 12        |
| <b>2</b>     | <b>REVISÃO BIBLIOGRÁFICA . . . . .</b>                             | <b>14</b> |
| 2.1          | DOENÇAS CARDIOVASCULARES . . . . .                                 | 14        |
| 2.2          | <i>MACHINE LEARNING</i> . . . . .                                  | 15        |
| <b>2.2.1</b> | <b>ETAPAS DE <i>MACHINE LEARNING</i> . . . . .</b>                 | <b>16</b> |
| <b>2.2.2</b> | <b>ALGORÍTIMOS DE <i>MACHINE LEARNING</i> . . . . .</b>            | <b>18</b> |
| 2.2.2.1      | REGRESSÃO LOGÍSTICA . . . . .                                      | 19        |
| 2.2.2.2      | <i>K-NEAREST NEIGHBORS - K-NN</i> . . . . .                        | 20        |
| 2.2.2.3      | <i>RANDOM FOREST</i> . . . . .                                     | 22        |
| <b>2.2.3</b> | <b>CRITÉRIOS DE AVALIAÇÃO DO MODELO . . . . .</b>                  | <b>23</b> |
| <b>2.2.4</b> | <b><i>PYTHON</i> PARA <i>MACHINE LEARNING</i> . . . . .</b>        | <b>25</b> |
| <b>3</b>     | <b>COLETA E PRÉ-PROCESSAMENTO DE DADOS . . . . .</b>               | <b>27</b> |
| 3.1          | ESCOLHA E CARACTERÍSTICAS DO BANCO DE DADOS . . . . .              | 27        |
| 3.2          | LIMPEZA E TRANSFORMAÇÃO DOS DADOS . . . . .                        | 30        |
| <b>4</b>     | <b>DIVISÃO DOS DADOS . . . . .</b>                                 | <b>33</b> |
| <b>5</b>     | <b>REGRESSÃO LOGÍSTICA . . . . .</b>                               | <b>35</b> |
| 5.1          | AVALIAÇÃO DO MODELO . . . . .                                      | 36        |
| <b>6</b>     | <b><i>K-NEAREST NEIGHBORS - K-NN</i> . . . . .</b>                 | <b>38</b> |
| 6.1          | AVALIAÇÃO DO MODELO . . . . .                                      | 40        |
| <b>7</b>     | <b><i>RANDOM FOREST</i> . . . . .</b>                              | <b>42</b> |
| 7.1          | AVALIAÇÃO DO MODELO . . . . .                                      | 44        |
| <b>8</b>     | <b>RESULTADOS . . . . .</b>                                        | <b>47</b> |
| <b>9</b>     | <b>CONCLUSÃO . . . . .</b>                                         | <b>49</b> |
|              | Referências . . . . .                                              | 50        |
|              | ANEXO A - AMOSTRA COMPLETA DO CONJUNTO DE DADOS ORIGINAL . . . . . | 53        |
|              | ANEXO B - CÓDIGO COMPLETO . . . . .                                | 54        |

## LISTA DE ILUSTRAÇÕES

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| Figura 1 – Diagrama das etapas do processo de <i>machine learning</i> . . . . . | 17 |
| Figura 2 – Exemplo ilustrativo para regressão logística. . . . .                | 20 |
| Figura 3 – Exemplo ilustrativo para <i>k-NN</i> . . . . .                       | 21 |
| Figura 4 – Diagrama <i>random forest</i> . . . . .                              | 23 |

## LISTA DE TABELAS

|                                                                                             |    |
|---------------------------------------------------------------------------------------------|----|
| Tabela 1 – Matriz de Confusão. . . . .                                                      | 24 |
| Tabela 2 – Amostra dos registros do conjunto de dados original. . . . .                     | 29 |
| Tabela 3 – Informações do conjunto de dados original. . . . .                               | 29 |
| Tabela 4 – Resumo estatístico dos dados originais. . . . .                                  | 31 |
| Tabela 5 – Resumo estatístico dos dados após a limpeza e transformação. . . . .             | 32 |
| Tabela 6 – Matriz de confusão regressão logística. . . . .                                  | 36 |
| Tabela 7 – Estatística do modelo regressão logística. . . . .                               | 37 |
| Tabela 8 – Matriz de confusão $k$ -NN. . . . .                                              | 40 |
| Tabela 9 – Estatística do modelo $k$ -NN. . . . .                                           | 40 |
| Tabela 10 – Matriz de Confusão <i>random forest</i> . . . . .                               | 44 |
| Tabela 11 – Estatística do modelo <i>random forest</i> . . . . .                            | 45 |
| Tabela 12 – Estatística dos 3 modelos. . . . .                                              | 47 |
| Tabela 13 – Amostra completa dos registros do conjunto de dados original (Parte 1). . . . . | 53 |
| Tabela 14 – Amostra completa dos registros do conjunto de dados original (Parte 2). . . . . | 53 |

# 1 INTRODUÇÃO

## 1.1 CONTEXTUALIZAÇÃO E MOTIVAÇÃO

eAs doenças cardiovasculares representam um desafio significativo para a saúde pública em todo o mundo, sendo responsáveis por uma parcela substancial de morbidade e mortalidade. De acordo com a Organização Mundial da Saúde (OMS), essas doenças são responsáveis por aproximadamente 17,9 milhões de mortes a cada ano, representando cerca de 31 % de todas as mortes globais (ORGANIZAÇÃO MUNDIAL DA SAÚDE, 2023).

A previsão e predição das doenças cardiovasculares têm um papel fundamental na identificação precoce de indivíduos em risco, permitindo intervenções preventivas e o manejo adequado da condição. Ao estimar a probabilidade de um indivíduo desenvolver problemas cardíacos no futuro com base em fatores de risco, histórico médico, exames laboratoriais e outros dados relevantes, é possível adotar medidas preventivas eficazes, como mudanças no estilo de vida, controle da pressão arterial, controle do diabetes, redução do colesterol e uso de medicamentos específicos, quando necessário.

Nesse contexto, o uso de técnicas de *machine learning* tem sido amplamente explorado, pois esses algoritmos são capazes de analisar grandes volumes de dados clínicos, genéticos e de estilo de vida, identificando padrões e associações que podem estar relacionados ao risco cardiovascular. Algoritmos de *machine learning*, como regressão logística, *random forest* e *k-Nearest Neighbors* (k-NN), podem ser treinados com conjuntos de dados representativos e, em seguida, aplicados para prever o risco individual de desenvolver doenças cardiovasculares.

Estudos têm demonstrado que as técnicas de *machine learning* podem melhorar a precisão da predição de doenças cardiovasculares em comparação com os modelos tradicionais. Essas abordagens podem levar em consideração uma ampla variedade de variáveis, incluindo características demográficas, dados clínicos, resultados de exames e informações genéticas, permitindo uma avaliação mais abrangente e personalizada do risco cardiovascular.

Por exemplo, um estudo conduzido por *Musa et al.* em 2020 utilizou técnicas de *machine learning* para prever o risco de doenças cardiovasculares em uma população de pacientes com diabetes tipo 2. Os resultados mostraram que os modelos de *machine learning* alcançaram uma maior precisão na predição do risco cardiovascular em comparação com os modelos tradicionais baseados em escores de risco convencionais (MUSA; ULLAH; BISWAS et al., 2020). Além disso, um estudo realizado por *Krittana Wong et al.* em 2019 investigou o uso de algoritmos de *machine learning* para prever o risco de eventos cardiovasculares em uma população geral. Os resultados indicaram que os modelos de *machine learning* foram capazes de identificar indivíduos de alto risco com maior precisão do que os modelos convencionais baseados em fatores de risco tradicionais (KRITTANAWONG;

JOHNSON; ROSENSON et al., 2019).

É importante ressaltar que, embora as técnicas de *machine learning* apresentem um grande potencial na área cardiovascular, elas não substituem a expertise médica. Esses algoritmos devem ser utilizados como ferramentas de suporte para auxiliar os profissionais de saúde na tomada de decisões clínicas.

## 1.2 OBJETIVOS

Neste trabalho, o objetivo principal é desenvolver e avaliar algoritmos de *machine learning* para três técnicas específicas: regressão logística, *random forest* e *k-Nearest Neighbors (k-NN)*. Esses algoritmos serão aplicados a um conjunto de dados disponível no *Kaggle* (KAGGLE, Acesso em 2023), uma plataforma amplamente utilizada para competições e projetos de ciência de dados.

Uma vez implementados os algoritmos, serão conduzidas análises comparativas para avaliar o desempenho de cada um deles. Serão consideradas métricas comuns, como acurácia, precisão, *recall* e *F1-score*, que fornecem uma visão abrangente sobre a eficácia dos algoritmos em relação à sua capacidade de fazer previsões corretas e lidar com diferentes classes ou categorias de dados.

Essas análises comparativas serão cruciais para entender e identificar quais algoritmos apresentam melhor desempenho no contexto do conjunto de dados em questão. Além disso, a avaliação das métricas de desempenho também fornecerá informações sobre a capacidade dos algoritmos de generalizar os resultados para novos dados, ou seja, se eles são capazes de realizar previsões precisas mesmo quando aplicados a dados que não foram utilizados durante o treinamento.

## 1.3 ESTRUTURA DO TRABALHO

O trabalho é dividido em 9 capítulos e 2 anexos. O capítulo 1 contém a introdução, que apresenta a motivação para o estudo de detecção e previsão de doenças cardiovasculares utilizando técnicas de *machine learning*.

No capítulo 2, é realizada uma revisão bibliográfica sobre doenças cardiovasculares, abordando sua epidemiologia, fatores de risco e impacto na saúde pública. Também são explorados conceitos fundamentais de *machine learning*, incluindo etapas, algoritmos e critérios de avaliação. Adicionalmente, destaca-se a utilização da linguagem *Python* para implementação de modelos de *machine learning*.

No capítulo 3, é descrito o processo de coleta e pré-processamento dos dados utilizados no estudo. São explicadas a escolha do banco de dados e suas características relevantes para a análise de doenças cardiovasculares. Além disso, são apresentadas as etapas de limpeza e transformação dos dados.

No capítulo 4, é discutida a divisão dos dados em conjuntos de treinamento e teste.

Os capítulos 5, 6 e 7 aborda a aplicação dos algoritmos de regressão logística, *k-Nearest Neighbors* e *random forest* na detecção de doenças cardiovasculares. Também são realizadas avaliações do modelo, considerando métricas de desempenho e a interpretação dos resultados.

O capítulo 8 apresenta os resultados obtidos ao aplicar os modelos de regressão logística, *k-Nearest Neighbors* e *random forest* no estudo de detecção e previsão de doenças cardiovasculares. São discutidos os principais achados e conclusões a partir desses resultados.

No capítulo 9, são apresentadas as conclusões finais do estudo. São resumidos os principais resultados alcançados na detecção e previsão de doenças cardiovasculares por meio das técnicas de *machine learning*.

Anexo A contém uma amostra completa do conjunto de dados utilizados.

Anexo B contém todo o código utilizado no trabalho.

## 2 REVISÃO BIBLIOGRÁFICA

### 2.1 DOENÇAS CARDIOVASCULARES

As doenças cardiovasculares são condições que afetam o coração e os vasos sanguíneos, podendo ter um impacto significativo na saúde e bem-estar das pessoas. Elas são uma das principais causas de morte em todo o mundo, causando um grande número de óbitos (CARDIOVASCULAR... , 2021).

Existem diferentes tipos de doenças cardiovasculares, como doenças das artérias coronárias, doenças cerebrovasculares, doenças dos vasos sanguíneos periféricos e doenças cardíacas congênitas. As doenças das artérias coronárias ocorrem quando as artérias que fornecem sangue ao coração ficam estreitadas ou bloqueadas, podendo causar dor no peito e ataques cardíacos. As doenças cerebrovasculares afetam os vasos sanguíneos que levam sangue ao cérebro, podendo levar a derrames e problemas neurológicos. As doenças dos vasos sanguíneos periféricos afetam as artérias e veias fora do coração e do cérebro, podendo causar problemas circulatórios nas pernas e braços. Já as doenças cardíacas congênitas são problemas estruturais do coração presentes desde o nascimento (AMERICAN HEART ASSOCIATION, 2021).

As doenças cardiovasculares têm impactos significativos nas pessoas afetadas. Elas podem levar a complicações graves, como insuficiência cardíaca e arritmias, afetando a capacidade do coração de bombear sangue adequadamente. Essas condições podem causar sintomas como falta de ar, fadiga e inchaço nas pernas. Além dos impactos físicos, as doenças cardiovasculares também podem ter um impacto emocional e social, afetando a qualidade de vida e a capacidade de realizar atividades diárias (AMERICAN HEART ASSOCIATION, 2021).

Além dos sintomas e complicações, as doenças cardiovasculares também têm um impacto econômico significativo. Elas geram altos custos com tratamentos médicos e hospitalares, além de resultar em perdas de produtividade devido a incapacidade e morte prematura. Esses custos afetam tanto os indivíduos como a sociedade como um todo (CARDIOVASCULAR... , 2021).

Para lidar com as doenças cardiovasculares, é importante adotar medidas de prevenção e controle. Isso inclui a adoção de um estilo de vida saudável, com uma dieta equilibrada, prática regular de exercícios físicos, não fumar e controlar fatores de risco, como hipertensão, diabetes e colesterol elevado. Além disso, é essencial buscar atendimento médico adequado para o diagnóstico precoce e tratamento das doenças cardiovasculares (CAUSES... , 2021).

A pesquisa contínua nessa área é fundamental para entender melhor as causas, os fatores de risco e desenvolver abordagens mais eficazes de prevenção e tratamento



das doenças cardiovasculares. Com mais conhecimento e melhores estratégias, pode-se reduzir o impacto dessas doenças e melhorar a saúde cardiovascular da população em geral (CARDIOVASCULAR... , 2021; AMERICAN HEART ASSOCIATION, 2021; CAUSES... , 2021).

## 2.2 MACHINE LEARNING

*Machine learning*, ou aprendizado de máquina, é uma área da ciência da computação que permite que os computadores aprendam e melhorem seu desempenho em determinadas tarefas por meio da análise de dados e padrões (MITCHELL, 1997). Essa área tem ganhado destaque nas últimas décadas devido ao seu potencial em diversos campos, desde ciência de dados e inteligência artificial até medicina, finanças e muito mais.

O objetivo fundamental do *machine learning*, é desenvolver algoritmos e modelos que possam aprender com os dados disponíveis, identificar padrões e tomar decisões ou fazer previsões sem a necessidade de serem explicitamente programados (MURPHY, 2012). Em vez de seguir instruções rígidas, os modelos de *machine learning*, são treinados para aprender com os exemplos e experiências passadas, permitindo que se adaptem e melhorem seu desempenho à medida que são expostos a mais dados.

Existem diferentes abordagens de *machine learning*, sendo as principais: aprendizado supervisionado, aprendizado não supervisionado e aprendizado por reforço (BISHOP, 2006). No aprendizado supervisionado, os modelos são treinados com dados rotulados, em que é fornecida uma resposta correta para cada exemplo. O objetivo é aprender a mapear os dados de entrada para as respostas corretas e, posteriormente, fazer previsões ou tomar decisões para novos dados não rotulados.

Já no aprendizado não supervisionado, não há rótulos nos dados de treinamento. O objetivo é descobrir padrões, estruturas ocultas e relações nos dados sem a orientação de respostas corretas. Essa abordagem é útil para tarefas como agrupamento de dados, redução de dimensionalidade e detecção de anomalias (UNSUPERVISED... , 2009).

No aprendizado por reforço, os modelos aprendem através de interações com um ambiente, recebendo recompensas ou penalidades para cada ação tomada. O objetivo é encontrar a sequência de ações que maximize a recompensa ao longo do tempo, levando em consideração a interação contínua entre o modelo e o ambiente (SUTTON; BARTO, 2018).

Os algoritmos de *machine learning*, são alimentados com conjuntos de dados, que consistem em exemplos de entrada (características) e as respostas corretas correspondentes. Esses dados são divididos em conjuntos de treinamento, validação e teste, permitindo que o modelo seja treinado, ajustado e avaliado em diferentes etapas (GERON, 2017).

As aplicações de *machine learning*, são amplas e estão presentes em nossa vida diária. Desde assistentes virtuais em *smartphones* até sistemas de recomendação em plataformas de *streaming*, essas tecnologias desempenham um papel fundamental na

análise de grandes volumes de dados, extração de informações valiosas e automatização de processos (AGGARWAL, 2018; HASTIE; TIBSHIRANI; FRIEDMAN, 2009).

No entanto, é importante destacar que o sucesso do *machine learning*, depende de uma combinação de dados de alta qualidade, algoritmos apropriados, recursos computacionais adequados e compreensão sólida dos problemas e contextos em que estão sendo aplicados. Além disso, questões éticas e de privacidade também são relevantes, exigindo uma abordagem responsável e cuidadosa ao lidar com dados sensíveis e tomar decisões automatizadas (BAROCAS; SELBST, 2020).

Conforme a pesquisa e a inovação em *machine learning*, continuam avançando, novos algoritmos, técnicas e abordagens estão sendo desenvolvidos para enfrentar desafios mais complexos e explorar o potencial total do aprendizado de máquina. Algoritmos como redes neurais artificiais, árvores de decisão, métodos de aprendizado profundo e algoritmos genéticos estão impulsionando avanços significativos em áreas como visão computacional, processamento de linguagem natural, reconhecimento de padrões, análise de dados e muito mais (GOODFELLOW; BENGIO; COURVILLE, 2016; RUSSELL; NORVIG, 2009).

No entanto, apesar dos benefícios e das oportunidades que o *machine learning*, oferece, também encontra-se desafios importantes. A interpretação dos resultados dos modelos, o viés nos dados de treinamento, a necessidade de lidar com grandes volumes de dados e a garantia da segurança e privacidade dos dados são questões cruciais que devem ser abordadas (ZHANG; LIU, 2021; BAROCAS; SELBST, 2020).

Além disso, a educação e o entendimento público sobre *machine learning*, também são fundamentais. A medida que a tecnologia continua a impactar várias esferas da nossa sociedade, é essencial promover a alfabetização em dados e a compreensão dos princípios básicos do aprendizado de máquina, a fim de capacitar as pessoas a tomar decisões informadas e éticas (DOMINGOS, 2018).

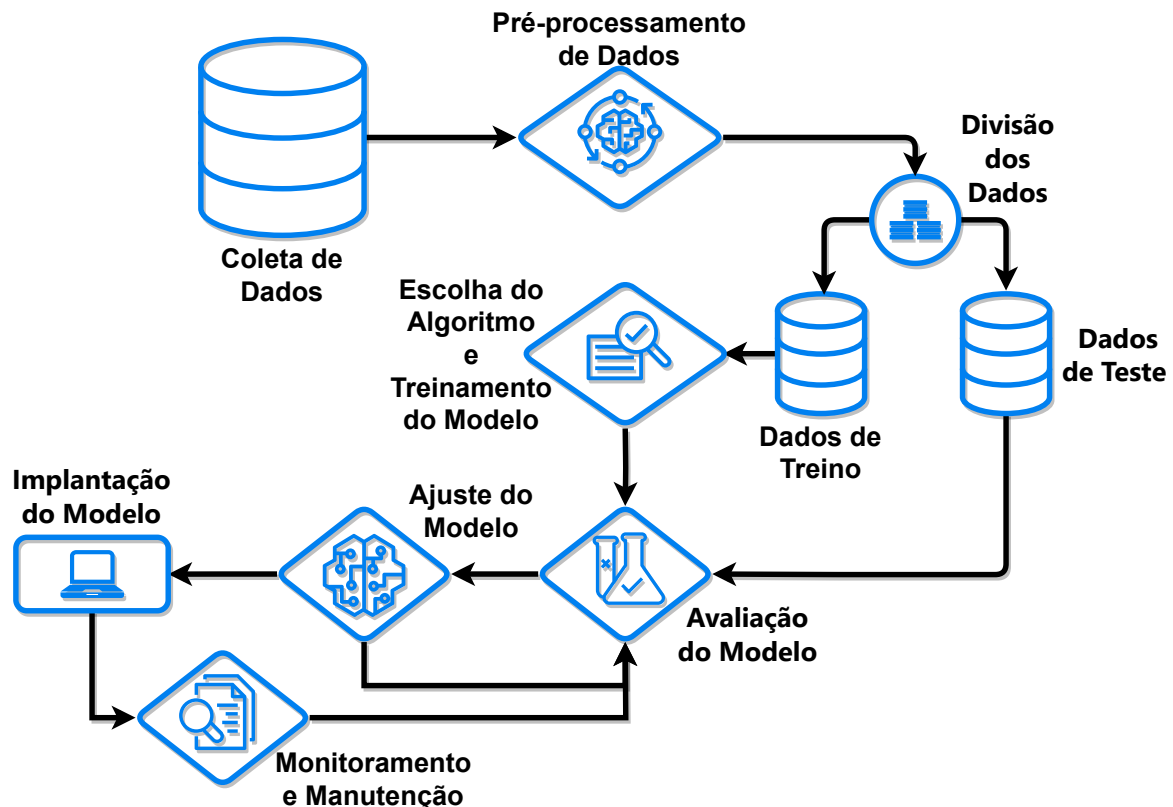
### 2.2.1 ETAPAS DE MACHINE LEARNING

O processo de *machine learning*, envolve várias etapas essenciais que permitem aos computadores aprender a partir de dados e tomar decisões ou fazer previsões como mostra a Figura 1. Essas etapas são fundamentais para o desenvolvimento e aplicação de modelos de *machine learning*, de forma eficaz.

**Coleta de Dados:** A primeira etapa do processo de *machine learning*, é a coleta de dados relevantes para o problema em questão. Isso pode envolver a obtenção de conjuntos de dados existentes, a criação de dados sintéticos ou até mesmo a coleta de dados em tempo real (HASTIE; TIBSHIRANI; FRIEDMAN, 2009).

**Pré-processamento de Dados:** Nesta etapa, os dados são preparados para serem utilizados pelo algoritmo de *machine learning*. Isso pode envolver a limpeza dos dados, tratamento de valores ausentes, normalização, codificação de variáveis categóricas, entre

**Figura 1** – Diagrama das etapas do processo de *machine learning*.



Fonte: Elaborado pelo autor (2023)

outras transformações necessárias (GERON, 2017).

**Divisão dos Dados:** Os dados são divididos em conjuntos de treinamento e teste. O conjunto de treinamento é utilizado para treinar o modelo de *machine learning*, enquanto o conjunto de teste é usado para avaliar o desempenho do modelo em dados não vistos anteriormente (AGGARWAL, 2018).

**Escolha do Algoritmo:** Com base no tipo de problema e nos dados disponíveis, é selecionado o algoritmo de *machine learning*, mais adequado. Existem diversos algoritmos disponíveis, como regressão linear, árvores de decisão, redes neurais, entre outros (SUTTON; BARTO, 2018).

**Treinamento do Modelo:** Nesta etapa, o modelo de *machine learning*, é treinado usando o conjunto de treinamento. O algoritmo utiliza os dados de treinamento para aprender os padrões e relações presentes nos dados (HASTIE; TIBSHIRANI; FRIEDMAN, 2009).

**Avaliação do Modelo:** Após o treinamento, o modelo é avaliado usando o conjunto de teste. Métricas de desempenho, como acurácia, precisão, *recall* e *F1-score*, são calculadas para medir a qualidade das previsões feitas pelo modelo (GERON, 2017).

**Ajuste do Modelo:** Se o desempenho do modelo não for satisfatório, é possível

realizar ajustes nos hiperparâmetros do algoritmo ou experimentar outros algoritmos para melhorar os resultados (HASTIE; TIBSHIRANI; FRIEDMAN, 2009).

**Implantação do Modelo:** Após o treinamento e ajuste do modelo, ele pode ser implantado em um ambiente de produção para fazer previsões em tempo real ou automatizar tarefas específicas (GERON, 2017).

**Monitoramento e Manutenção:** É importante monitorar o desempenho do modelo em produção e realizar manutenção regularmente. Isso envolve a verificação contínua da precisão das previsões, atualização do modelo com novos dados e adaptação a possíveis mudanças no ambiente (HASTIE; TIBSHIRANI; FRIEDMAN, 2009).

### 2.2.2 ALGORÍTIMOS DE *MACHINE LEARNING*

Algoritmos de *machine learning* são ferramentas computacionais fundamentais que desempenham um papel crucial na construção de sistemas inteligentes. Esses algoritmos possibilitam que os sistemas aprendam a partir de dados, identificando padrões e relações complexas, e realizem tarefas como tomar decisões e fazer previsões, sem a necessidade de programação explícita (HASTIE; TIBSHIRANI; FRIEDMAN, 2009; MURPHY, 2012; BISHOP, 2006).

No processo de *machine learning*, os algoritmos são projetados para processar grandes volumes de dados, extrair características relevantes e criar modelos capazes de generalizar a partir desses dados para novos exemplos. Esses modelos são alimentados com dados de treinamento, nos quais são ajustados e otimizados para aprender com os padrões presentes nos dados (HASTIE; TIBSHIRANI; FRIEDMAN, 2009).

Uma vez que os algoritmos de *machine learning* são capazes de lidar com uma ampla gama de problemas e domínios, eles têm sido aplicados em diversas áreas, como reconhecimento de padrões, processamento de linguagem natural, visão computacional, análise de dados, entre outros. A capacidade de aprender com os dados e adaptar-se a novas informações torna os algoritmos de *machine learning* extremamente poderosos e versáteis (MURPHY, 2012; BISHOP, 2006).

Ao buscar na literatura, foram considerados diversos métodos de classificação disponíveis, levando em conta suas características e aplicabilidade aos dados e objetivos do trabalho. Dentre as opções analisadas, três métodos foram escolhidos por se destacarem como soluções promissoras, ao mesmo tempo em que possuem uma implementação relativamente simples. A escolha de métodos de simples implementação é importante para facilitar a compreensão e replicação dos resultados obtidos no trabalho. Além disso, essa abordagem permite uma maior agilidade na implementação dos modelos de classificação, tornando o processo mais eficiente.

Nas seções a seguir, serão apresentadas de forma detalhada as técnicas de regressão logística, *K-Nearest Neighbors* - *k-NN* e *random forest*, que foram escolhidas para o

trabalho de classificação proposto.

### 2.2.2.1 REGRESSÃO LOGÍSTICA

A regressão logística é um algoritmo de *machine learning* amplamente utilizado para resolver problemas de classificação (HASTIE; TIBSHIRANI; FRIEDMAN, 2009). Enquanto a regressão linear é usada para prever valores numéricos contínuos, a regressão logística é projetada especificamente para prever a probabilidade de um evento ocorrer, atribuindo uma classe ou rótulo a uma instância de dados.

A regressão logística é frequentemente aplicada quando o problema de classificação é binário, ou seja, existem apenas duas classes possíveis (HASTIE; TIBSHIRANI; FRIEDMAN, 2009). No entanto, ela também pode ser estendida para problemas de classificação multiclasse. O objetivo principal da regressão logística é encontrar a relação entre as variáveis de entrada (características) e a variável de saída (classe), estimando os coeficientes que melhor se ajustam aos dados de treinamento.

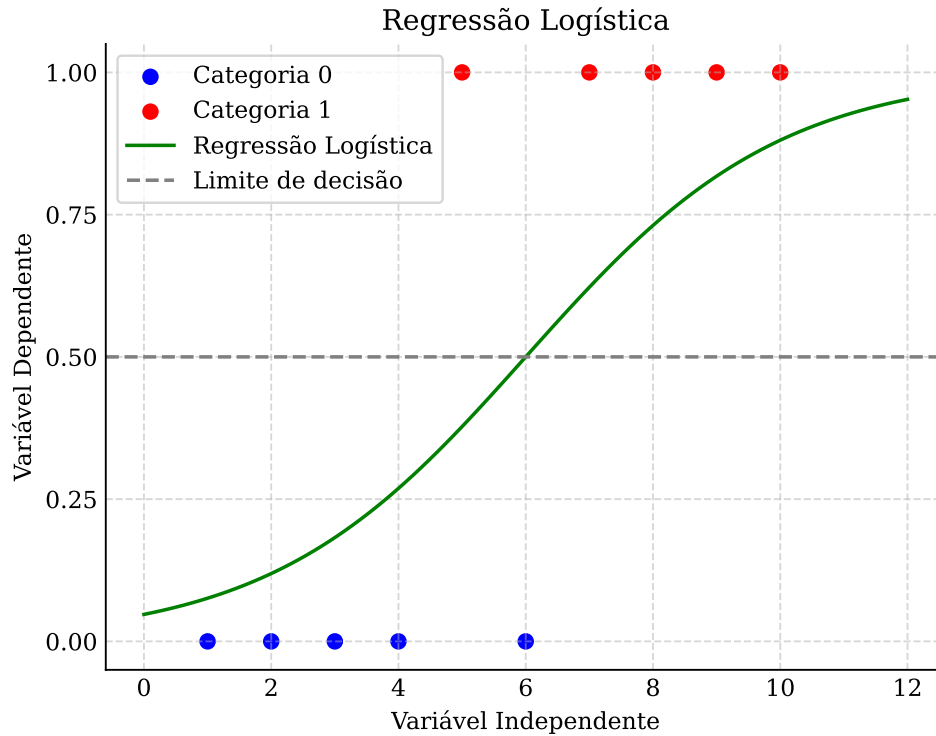
A regressão logística utiliza a função logística, também conhecida como função sigmoide, para modelar a relação entre as variáveis de entrada e a probabilidade de pertencer a uma determinada classe (HASTIE; TIBSHIRANI; FRIEDMAN, 2009). A função sigmoide mapeia um valor real para um valor entre 0 e 1, o que representa a probabilidade de pertencer à classe positiva. A função sigmoide é definida conforme a Equação (2.1):

$$f(x) = \frac{1}{1 + e^{-x}} \quad (2.1)$$

Nessa equação, o valor de  $x$  é a combinação linear das características de entrada multiplicadas pelos coeficientes correspondentes. Essa combinação linear é então aplicada à função sigmoide para obter a probabilidade.

Durante o treinamento da regressão logística, os coeficientes são ajustados iterativamente por meio de um algoritmo de otimização, como o Gradiente Descendente (HASTIE; TIBSHIRANI; FRIEDMAN, 2009), para minimizar uma função de perda, como a função de log-verossimilhança. O objetivo é encontrar os coeficientes que maximizam a probabilidade de observar os rótulos corretos nos dados de treinamento.

Uma vez que o modelo de regressão logística é treinado, ele pode ser usado para fazer previsões em novos dados, atribuindo probabilidades às classes. Geralmente, um limiar é escolhido para determinar a classe final com base nas probabilidades previstas. Por exemplo, se a probabilidade de pertencer à classe positiva for maior que o limiar, a instância de dados será classificada como positiva, caso contrário, será classificada como negativa, a Figura 2 ilustra uma aplicação.

**Figura 2** – Exemplo ilustrativo para regressão logística.

**Fonte:** Elaborado pelo autor (2023)

A regressão logística é amplamente utilizada em diversas aplicações, como detecção de spam, diagnóstico médico, análise de sentimentos, entre outros (BISHOP, 2006). Ela é um dos algoritmos mais populares e a interpretação dos coeficientes pode fornecer informações valiosas sobre a importância relativa das características na determinação da classe.

#### 2.2.2.2 *K-NEAREST NEIGHBORS - K-NN*

O algoritmo *K-Nearest Neighbors - k-NN* é uma técnica poderosa e amplamente utilizada no campo do aprendizado de máquina (HASTIE; TIBSHIRANI; FRIEDMAN, 2009; BISHOP, 2006; MURPHY, 2012). É considerado um algoritmo de aprendizado supervisionado, onde se busca classificar ou prever valores com base em exemplos rotulados previamente. O *k-NN* é conhecido por sua simplicidade e eficácia, sendo aplicado em diversas áreas, como reconhecimento de padrões, processamento de imagens, bioinformática e sistemas de recomendação.

O princípio fundamental do *k-NN* é baseado na ideia de que objetos semelhantes tendem a estar próximos uns dos outros no espaço de características. Para determinar a proximidade entre os exemplos, o algoritmo utiliza uma medida de distância, comumente a distância Euclidiana (HASTIE; TIBSHIRANI; FRIEDMAN, 2009). Essa medida calcula a distância entre dois pontos em um espaço  $n$ -dimensional, considerando as diferenças em cada dimensão. Matematicamente, a distância Euclidiana é definida como:

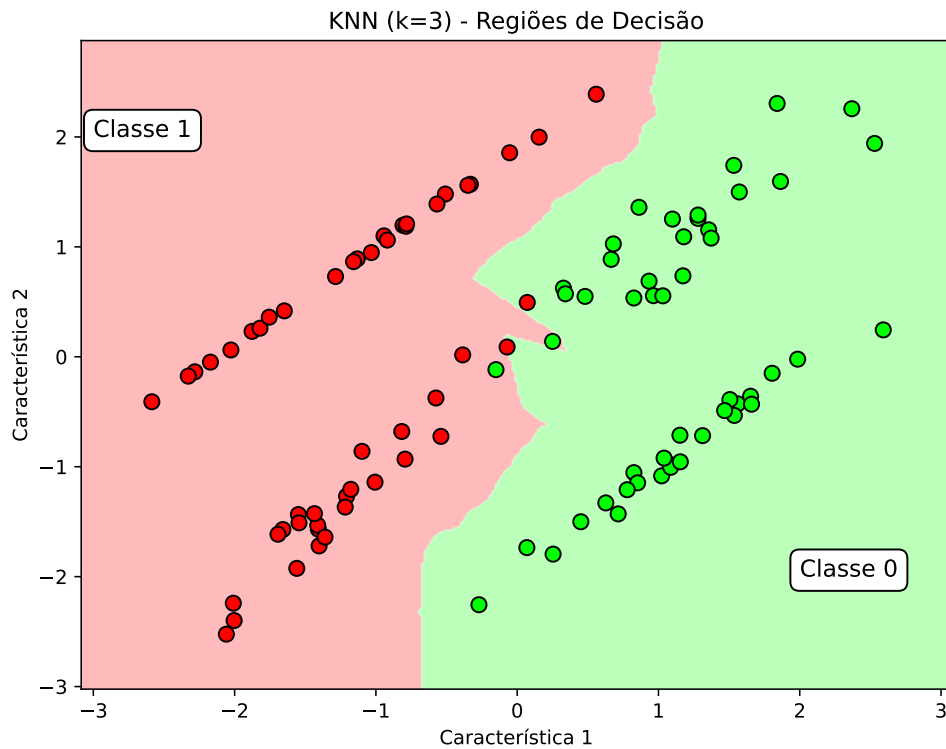
$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (2.2)$$

Onde  $x$  e  $y$  são pontos no espaço de características, e  $x_i$  e  $y_i$  são as coordenadas desses pontos em cada dimensão.

O algoritmo  $k$ -NN opera em duas fases principais: a fase de treinamento e a fase de teste. Durante a fase de treinamento, o algoritmo armazena os exemplos de treinamento e seus respectivos rótulos. Na fase de teste, quando um novo exemplo é apresentado, o algoritmo calcula a distância entre esse exemplo e todos os exemplos de treinamento. Em seguida, seleciona os  $K$  vizinhos mais próximos com base nessas distâncias.

No caso de problemas de classificação, a classe mais comum entre os  $K$  vizinhos é atribuída ao novo exemplo como ilustra a Figura 3. Já em problemas de regressão, a média dos valores de saída dos  $K$  vizinhos mais próximos é considerada como a previsão para o novo exemplo.

**Figura 3** – Exemplo ilustrativo para  $k$ -NN.



**Fonte:** Elaborado pelo autor (2023)

Um aspecto importante na aplicação do  $k$ -NN é a escolha adequada do valor  $K$ , conhecido como hiperparâmetro (HASTIE; TIBSHIRANI; FRIEDMAN, 2009). Valores pequenos de  $K$  tornam o modelo mais sensível a ruídos nos dados, podendo resultar em classificações ou previsões instáveis. Por outro lado, valores grandes de  $K$  podem levar à perda de informações locais relevantes, resultando em um viés maior do modelo.

Embora o  $k$ -NN seja um algoritmo relativamente simples, ele possui suas limitações. A principal desvantagem é o custo computacional durante a fase de classificação, especialmente quando se lida com grandes conjuntos de dados. Além disso, o  $k$ -NN assume que os atributos dos exemplos são igualmente importantes, o que pode não ser verdade em todas as situações.

### 2.2.2.3 RANDOM FOREST

O algoritmo *random forest* (Floresta Aleatória) é um método de aprendizado de máquina amplamente utilizado que combina a força de múltiplas árvores de decisão para realizar tarefas de classificação e regressão (BREIMAN, 2001; LIAW; WIENER, 2002). Essa abordagem se destaca pela sua capacidade de lidar com conjuntos de dados complexos, de alta dimensionalidade e com problemas de *overfitting*.

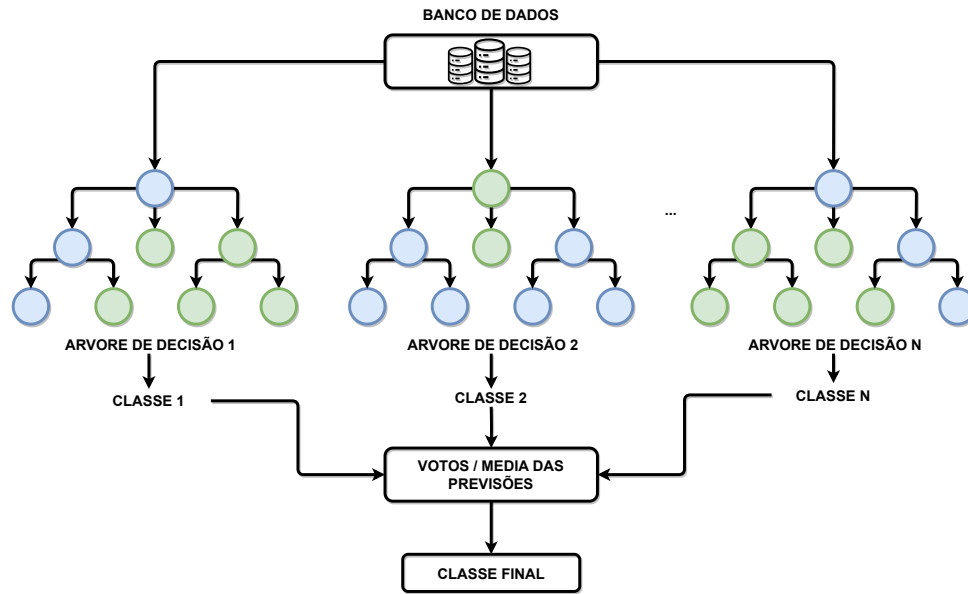
A principal ideia por trás do *random forest* é a criação de um conjunto diversificado de árvores de decisão, que são treinadas independentemente em diferentes subconjuntos dos dados de treinamento. Cada árvore é construída de forma aleatória, tanto na seleção das amostras de treinamento como nas características utilizadas em cada divisão da árvore (BREIMAN, 2001). Essa aleatoriedade introduzida no processo de construção das árvores é crucial para reduzir a variância e evitar o superajuste (*overfitting*).

No caso da classificação, o *random forest* realiza uma votação das previsões de cada árvore para determinar a classe final da amostra. Já na regressão, o *random forest* realiza uma média das previsões de cada árvore para obter o valor final da resposta (LIAW; WIENER, 2002). Essa combinação de previsões de várias árvores aumenta a estabilidade e a robustez do modelo, tornando-o mais capaz de generalizar bem para novos dados.

Uma das vantagens do *random forest* é a sua capacidade de medir a importância relativa das características utilizadas na construção das árvores. Isso é possível através do cálculo da diminuição média da impureza (ou ganho de informação) causada por cada característica (LIAW; WIENER, 2002). Essa medida de importância permite uma melhor compreensão das características mais relevantes para a tarefa em questão.

Em termos matemáticos, a equação específica para o *random forest* é complexa, pois envolve a combinação das previsões de várias árvores. Cada árvore de decisão é construída usando critérios como o índice de *Gini* ou a entropia para realizar as divisões dos nós (BREIMAN, 2001). A previsão final do *random forest* é obtida por meio de uma agregação de votos ou médias, dependendo da tarefa em questão como mostra a Figura 4.



**Figura 4** – Diagrama *random forest*.

**Fonte:** Elaborado pelo autor (2023)

### 2.2.3 CRITÉRIOS DE AVALIAÇÃO DO MODELO

As métricas de avaliação desempenham um papel crucial na análise do desempenho e eficácia dos modelos de *machine learning*. Entre as métricas comumente utilizadas, destacam-se a acurácia, precisão, *recall* e *F1-score*, que fornecem uma visão abrangente da capacidade do modelo em fazer previsões corretas e lidar com diferentes classes ou categorias de dados (ZHANG; OU-YANG, 2018).

Além dessas métricas, uma ferramenta importante para avaliar o desempenho de um modelo de classificação é a matriz de confusão (GALAR et al., 2012). A matriz de confusão permite visualizar as previsões feitas pelo modelo em relação às classes reais. Ela fornece uma tabela que mostra a contagem de amostras classificadas corretamente e incorretamente para cada classe, como mostra a Tabela 1.

Na matriz de confusão, as classes reais são representadas nas linhas da tabela, enquanto as previsões do modelo são representadas nas colunas. Cada célula da tabela representa a contagem de amostras que foram classificadas de acordo com a interseção entre a classe real e a classe prevista.

As principais informações presentes na matriz de confusão são:

- Verdadeiro Positivo (VP): Representa o número de amostras que foram corretamente classificadas como positivas.
- Falso Positivo (FP): Indica o número de amostras que foram erroneamente classificadas como positivas, quando na verdade pertencem à classe negativa.

- Falso Negativo (FN): Refere-se ao número de amostras que foram erroneamente classificadas como negativas, quando na verdade pertencem à classe positiva.
- Verdadeiro Negativo (VN): Representa o número de amostras que foram corretamente classificadas como negativas.

**Tabela 1** – Matriz de Confusão.

| Positivo                 | Negativo                 |
|--------------------------|--------------------------|
| Verdadeiro Positivo (VP) | Falso Positivo (FP)      |
| Falso Negativo (FN)      | Verdadeiro Negativo (VN) |

A partir dessa matriz, é possível calcular diversas métricas de desempenho, incluindo a acurácia, precisão, *recall* e *F1-score*.

A acurácia é uma métrica básica que mede a proporção de previsões corretas em relação ao total de amostras (JAPKOWICZ; SHAH, 2011). Ela fornece uma medida geral da taxa de acerto do modelo, sendo calculada pela divisão do número de amostras corretamente previstas pelo total de amostras (POWERS, 2021), como mostra a Equação (2.3).

$$\text{Acurácia} = \frac{\text{Verdadeiros Positivos} + \text{Verdadeiros Negativos}}{\text{Total de Amostras}} \quad (2.3)$$

No entanto, a acurácia pode ser enganosa quando as classes estão desbalanceadas, ou seja, quando algumas classes têm um número significativamente maior ou menor de amostras do que outras (GALAR et al., 2012). Para lidar com essa limitação, outras métricas como precisão, *recall* e *F1-score* são utilizadas.

A precisão é a proporção de verdadeiros positivos (amostras corretamente previstas como positivas) em relação ao total de amostras previstas como positivas (JAPKOWICZ; SHAH, 2011). Ela é especialmente útil quando o foco está em minimizar os falsos positivos, ou seja, quando é importante evitar previsões errôneas de uma classe específica (POWERS, 2021). A precisão é calculada dividindo o número de verdadeiros positivos pelo somatório dos verdadeiros positivos e falsos positivos (POWERS, 2021), como mostra a Equação (2.4).

$$\text{Precisão} = \frac{\text{Verdadeiros Positivos}}{\text{Verdadeiros Positivos} + \text{Falsos Positivos}} \quad (2.4)$$

O *recall*, também conhecido como sensibilidade, é a proporção de verdadeiros positivos em relação ao total de amostras verdadeiramente positivas (JAPKOWICZ; SHAH, 2011). Ele é relevante quando o objetivo é minimizar os falsos negativos, ou seja, evitar deixar de identificar amostras de uma classe específica (POWERS, 2021). O *recall* é calculado

dividindo o número de verdadeiros positivos pelo somatório dos verdadeiros positivos e falsos negativos (POWERS, 2021), como mostra a Equação (2.5).

$$\text{Recall} = \frac{\text{Verdadeiros Positivos}}{\text{Verdadeiros Positivos} + \text{Falsos Negativo}} \quad (2.5)$$

O *F1-score* é uma medida que combina a precisão e o *recall* em uma única métrica (JAPKOWICZ; SHAH, 2011). É a média harmônica dessas duas métricas e fornece uma medida equilibrada do desempenho do modelo (POWERS, 2021). O *F1-score* é especialmente útil quando as classes estão desbalanceadas, pois leva em consideração tanto os falsos positivos quanto os falsos negativos (POWERS, 2021). O cálculo do *F1-score* é realizado pela multiplicação da precisão e *recall*, dividida pela soma da precisão e *recall*, multiplicada por dois (POWERS, 2021), como mostra a Equação (2.6).

$$F1\text{-score} = 2 \times \frac{\text{Precisão} \times \text{Recall}}{\text{Precisão} + \text{Recall}} \quad (2.6)$$

Essas métricas são amplamente utilizadas para avaliar o desempenho de modelos de *machine learning* e fornecem uma visão abrangente sobre a eficácia do algoritmo em fazer previsões corretas (ZHANG; OU-YANG, 2018). Ao considerar essas métricas em conjunto, é possível ter uma compreensão mais completa do desempenho do modelo e selecionar o modelo mais adequado para o problema em questão. É importante ressaltar que a escolha das métricas de avaliação depende do contexto e dos objetivos específicos do projeto.

#### 2.2.4 PYTHON PARA MACHINE LEARNING

Existem várias linguagens de programação amplamente utilizadas em projetos de *machine learning*, como *python*, *R*, *Java*, *C++* e *MATLAB*. Essas linguagens oferecem uma variedade de bibliotecas e ferramentas especializadas para o desenvolvimento de modelos de *machine learning*. A escolha da linguagem depende das necessidades e preferências dos desenvolvedores, assim como das características específicas do projeto em questão, para esse trabalho foi escolhida a linguagem *python* por ser uma linguagem de domínio do autor.

*Python* desempenha um papel significativo e cada vez mais relevante no campo de *machine learning*, consolidando-se como uma das linguagens de programação mais populares e amplamente utilizadas nessa área (ZHANG; OU-YANG, 2018). Sua popularidade é impulsionada por várias características e contribuições que *python* oferece para o desenvolvimento e implantação de modelos de *machine learning*.

Historicamente, *python* emergiu como uma linguagem de programação poderosa e versátil, ganhando destaque na comunidade de ciência de dados (GALAR et al., 2012). Sua sintaxe simples, legibilidade do código e facilidade de uso tornam *python* uma escolha ideal para desenvolvedores e cientistas de dados que buscam implementar algoritmos de *machine learning* de forma eficiente e eficaz.

Uma das principais vantagens de *python* para o campo de *machine learning* é o seu rico ecossistema de bibliotecas especializadas (ZHANG; OU-YANG, 2018). Bibliotecas populares, como *NumPy*, *pandas*, *scikit-learn*, *TensorFlow* e *PyTorch*, oferecem funcionalidades avançadas para manipulação de dados, pré-processamento, treinamento e avaliação de modelos. Essas bibliotecas simplificam o processo de implementação de algoritmos complexos, permitindo que os desenvolvedores se concentrem mais nos aspectos analíticos e de modelagem (GALAR et al., 2012).

Além disso, *python* se beneficia de uma vasta comunidade de desenvolvedores e cientistas de dados, resultando em uma ampla disponibilidade de recursos, tutoriais, exemplos de código e suporte online (ZHANG; OU-YANG, 2018). Fóruns, grupos de discussão e comunidades online fornecem um ambiente colaborativo e de aprendizado, permitindo que os usuários de *python* obtenham ajuda, compartilhem conhecimentos e se engajem em projetos de *machine learning*.

Outra vantagem do *python* é a sua capacidade de integração com outras tecnologias e *frameworks* (GALAR et al., 2012). *python* pode ser facilmente combinado com linguagens como C++ e Java, permitindo tirar proveito do desempenho e da eficiência dessas linguagens quando necessário. Além disso, o *python* é frequentemente utilizado em conjunto com ferramentas de visualização, como *Matplotlib* e *Seaborn*, facilitando a criação de gráficos e visualizações para análise de dados e resultados de *machine learning*.

Atualmente, *python* continua evoluindo e desempenhando um papel fundamental no desenvolvimento de soluções de *machine learning* (ZHANG; OU-YANG, 2018). Novas bibliotecas e frameworks são constantemente desenvolvidos, expandindo ainda mais as capacidades e possibilidades da linguagem. A comunidade *python* continua a crescer, impulsionando a inovação e promovendo a colaboração entre especialistas em *machine learning* (GALAR et al., 2012).

Em resumo, *python* é uma linguagem de programação essencial para o campo de *machine learning*, oferecendo um ambiente amigável, um vasto conjunto de bibliotecas especializadas e uma comunidade ativa de desenvolvedores. Sua versatilidade, facilidade de uso e integração com outras tecnologias contribuem para a sua popularidade e sucesso contínuo no contexto da ciência de dados e do *machine learning* (ZHANG; OU-YANG, 2018; GALAR et al., 2012).

### 3 COLETA E PRÉ-PROCESSAMENTO DE DADOS

A seleção de dados apropriados e o pré-processamento são etapas cruciais em projetos de *demachine learning*, que impactam diretamente a qualidade dos modelos e a precisão das previsões. A seleção de dados envolve a escolha de conjuntos de dados relevantes e representativos para treinar e avaliar os modelos. Já o pré-processamento refere-se à limpeza, transformação e normalização dos dados.

No processo de limpeza de dados, é necessário lidar com valores ausentes, *outliers* e ruídos. Os valores ausentes podem ser preenchidos ou as amostras correspondentes podem ser removidas para garantir a integridade dos dados utilizados na modelagem. Os *outliers*, que são pontos de dados que se desviam significativamente do padrão geral, devem ser identificados e tratados adequadamente para evitar distorções e impactos negativos nos modelos. Ruídos indesejados também precisam ser detectados e removidos, visando preservar a qualidade dos dados.

A transformação e normalização dos dados são etapas importantes para melhorar a distribuição, escala e interpretação dos dados. A normalização é comumente usada para ajustar os valores das variáveis em uma escala comum, facilitando a comparação e o treinamento dos modelos.

#### 3.1 ESCOLHA E CARACTERÍSTICAS DO BANCO DE DADOS

A escolha de um bom conjunto de dados é crucial para projetos de *demachine learning*. Nesse sentido, optou-se por utilizar um banco de dados disponível no *Kaggle*, uma plataforma reconhecida e amplamente utilizada para competições e projetos de ciência de dados. O *Kaggle* oferece conjuntos de dados de alta qualidade, abrangendo várias áreas temáticas e acompanhados de informações relevantes. Ao utilizar um banco de dados do *Kaggle*, economiza-se tempo na coleta e preparação dos dados, permitindo focar no desenvolvimento do modelo de *demachine learning*. Essa escolha justifica-se pela reputação, diversidade e facilidade de acesso oferecidas pelo *Kaggle*.

O banco de dados escolhido para este estudo é o "*Cardiovascular Disease Dataset*", que consiste em 70.000 registros de dados de pacientes. Esses registros foram coletados durante exames médicos e fornecem informações relevantes sobre 12 características distintas. As informações do banco de dados estão organizadas em 13 colunas, que são as seguintes:

- Coluna 1: Id
  - Funciona como índice, não tem informação relevante.
- Coluna 2: Idade
  - Característica objetiva e foi registrada em dias.

- Coluna 3: Altura
  - Característica objetiva e foi registrada em centímetros.
- Coluna 4: Peso
  - Característica objetiva e foi registrado em quilogramas.
- Coluna 5: Gênero
  - Característica objetiva; registro: 1 para masculino e 2 para feminino.
- Coluna 6: Pressão arterial sistólica
  - Característica de exame e foi registrada em milímetros de mercúrio.
- Coluna 7: Pressão arterial diastólica
  - Característica de exame e foi registrada em milímetros de mercúrio.
- Coluna 8: Colesterol
  - Característica de exame; registro: 1 para normal, 2 para acima do normal e 3 para muito acima do normal.
- Coluna 9: Glicose
  - Característica de exame; registro: 1 para normal, 2 para acima do normal e 3 para muito acima do normal
- Coluna 10: Fumante
  - Característica subjetiva e foi registrado: 1 para fumante e 0 para não fumante.
- Coluna 11: Consumo de álcool
  - Característica subjetiva; registro: 1 para consumo de álcool e 0 para não consumo de álcool.
- Coluna 12: Atividade física.
  - Característica subjetiva; registro: 1 para pratica de atividade física , 0 para não pratica de atividade física.
- Coluna 13: Doença cardiovascular
  - Característica de exame; registro: 1 para presença de doença cardiovascular e 0 para ausência de doença cardiovascular.

Para ilustrar os dados, a Tabela 2 mostra uma amostra dos registros do conjunto de dados. Uma amostra completa dos registros está disponível no Anexo A.

**Tabela 2** – Amostra dos registros do conjunto de dados original.

| Id | Idade | Gênero | Altura | Peso | ... | Doença cardiovascular |
|----|-------|--------|--------|------|-----|-----------------------|
| 0  | 18393 | 2      | 168    | 62,0 | ... | 0                     |
| 1  | 20228 | 1      | 156    | 85,0 | ... | 1                     |
| 2  | 18857 | 1      | 165    | 64,0 | ... | 1                     |
| 3  | 17623 | 2      | 169    | 82,0 | ... | 1                     |
| 4  | 17474 | 1      | 156    | 56,0 | ... | 0                     |

Para obter uma melhor compreensão do formato dos dados, o seguinte código pode foi utilizado:

```

1 # Carrega o arquivo CSV em um DataFrame
2 dados = pd.read_csv("/content/cardio_train.csv", sep=";")
3 # Exibe informações sobre o DataFrame
4 print(dados.info())

```

**Tabela 3** – Informações do conjunto de dados original.

| Coluna                      | Contagem de Não Nulos | Tipo    | Unidade                |
|-----------------------------|-----------------------|---------|------------------------|
| Id                          | 70000                 | int64   | -                      |
| Idade                       | 70000                 | int64   | Dias                   |
| Genero                      | 70000                 | int64   | 1/2                    |
| Altura                      | 70000                 | int64   | Centímetros            |
| Peso                        | 70000                 | float64 | Quilogramas            |
| Pressao_arterial_sistolica  | 70000                 | int64   | Milímetros de Mercúrio |
| Pressao_arterial_diastolica | 70000                 | int64   | Milímetros de Mercúrio |
| Colesterol                  | 70000                 | int64   | 1/2/3                  |
| Glicose                     | 70000                 | int64   | 1/2/3                  |
| Fumante                     | 70000                 | int64   | 0/1                    |
| Consumo_de_alcool           | 70000                 | int64   | 0/1                    |
| Atividade_fisica            | 70000                 | int64   | 0/1                    |
| Doenca_cardiovascular       | 70000                 | int64   | 0/1                    |

Como resultado, tem-se a Tabela 3, onde cada coluna tem o seguinte significado:

- Coluna: Nome da coluna.
- Contagem de Não Nulos: Contagem de valores não nulos na coluna, ou seja, a quantidade de valores presentes.
- Tipo: Tipo de dado da coluna.
  - *int64*: É um tipo numérico inteiro de 64 bits. Ele é usado para representar números inteiros sem parte decimal.
  - *float64*: É um tipo numérico de ponto flutuante de precisão dupla. Ele é usado para representar números reais com parte decimal.
- Unidade: Unidade que o dado foi registrado na tabela.

### 3.2 LIMPEZA E TRANSFORMAÇÃO DOS DADOS

Antes de realizar a limpeza e a transformação dos dados, foi adicionada uma nova coluna ao *DataFrame*. Essa coluna corresponderá ao Índice de Massa Corporal (IMC). O IMC é calculado dividindo o peso em quilogramas pela altura, dada em metros, elevada ao quadrado. Essa nova coluna será útil para analisar a composição corporal dos indivíduos, cujos dados estão presentes no *DataFrame*.

Para iniciar o processo de limpeza, transformação e normalização dos dados, foram removidas possíveis linhas duplicadas e a coluna "*Id*" do *DataFrame*, uma vez que essa coluna é considerada irrelevante, pois possui apenas a função de identificação e não contém informações relevantes para análise. Removê-la permitirá uma manipulação mais eficiente dos dados restantes.

Foi necessário realizar a transformação da coluna "*Idade*" de dias para anos. Essa mudança é benéfica, pois reduz a escala dos dados e simplifica a sua interpretação. Ao converter os dias em anos, torna-se mais fácil comparar a idade de diferentes indivíduos e facilita a análise geral do conjunto de dados. Essas duas ações foram realizadas com o código abaixo:

```

1 # Remover a coluna "id" do DataFrame
2 dados.drop("id", axis=1, inplace=True)
3 # Remover linhas duplicadas do DataFrame
4 dados.drop_duplicates(inplace=True)
5 # Converter a coluna "Idade" de dias para anos
6 dados['Idade'] = dados['Idade'].apply(lambda x:
    timedelta(days=x) / timedelta(days=365.25))
7 # Calcular o Índice de Massa Corporal (IMC) e adicionar uma nova
    coluna "IMC" ao DataFrame
8 dados["IMC"] = dados["Peso"] / (dados["Altura"]/100)**2

```



Para ter uma visão geral estatística dos dados foi usado o método "*describe*" que calcula várias medidas estatísticas, como contagem, média, desvio padrão, quartis e valores mínimo e máximo, como mostra o código abaixo:

```
1 dados.describe()
```

Os valores retornados mais relevantes de serem analisados pelo método podem ser vistos na Tabela 4:

**Tabela 4** – Resumo estatístico dos dados originais.

|               | IMC        | Pressão arterial<br>sistólica | Pressão arterial<br>diastólica |
|---------------|------------|-------------------------------|--------------------------------|
| Contagem      | 69976      | 69976                         | 69976                          |
| Media         | 27,55761   | 128,820453                    | 96,636261                      |
| Desvio Padrão | 6,09221    | 154,037729                    | 188,504581                     |
| Mínimo        | 3,471784   | -150,000000                   | -70,000000                     |
| Máximo        | 298,666667 | 16020,000000                  | 11000,000000                   |

Ao analisar a tabela, é evidente que existem valores máximos e mínimos que estão fora da realidade esperada. Esses valores extremos podem impactar diretamente nos resultados e na interpretação dos dados. Portanto, é recomendado estabelecer limites aceitáveis para o IMC, pressão arterial sistólica e diastólica, removendo os dados que estão além desses limites. Dessa forma, garante-se uma análise mais precisa e consistente dos dados.

Ao considerar valores extremos para o IMC, é possível estabelecer limites muito altos e muito baixos. Por exemplo, alguns estudos têm adotado um limite máximo extremo de 100 para indicar valores extremamente altos do IMC. Da mesma forma, um limite mínimo extremo, como 5, pode ser utilizado para representar valores extremamente baixos. No entanto, é fundamental ressaltar que esses valores estão fora da faixa realista de variação do IMC e são raros na prática clínica (WORLD HEALTH ORGANIZATION, 2022).

No caso da pressão arterial sistólica, o limite superior extremo de 220 mmHg indica um valor muito elevado, que pode indicar uma pressão arterial muito alta e potencialmente perigosa. Já o limite inferior extremo de 60 mmHg indica um valor muito baixo, que pode sugerir uma pressão arterial anormalmente baixa (ASSOCIATION, 2023; EUROPEAN SOCIETY OF CARDIOLOGY, 2023).

Para a pressão arterial diastólica, o limite superior extremo de 140 mmHg também indica um valor muito elevado, que pode indicar hipertensão arterial. O limite inferior extremo de 30 mmHg, por sua vez, representa um valor muito baixo, que pode ser indicativo de hipotensão arterial (ASSOCIATION, 2023; EUROPEAN SOCIETY OF CARDIOLOGY, 2023).

Para aplicar os filtros foi utilizado o seguinte código:

```

1 # Criação do filtro para selecionar linhas com base em condições
   específicas
2 filtro = ((dados["Pressao_arterial_sistolica"] > 220) |
3           (dados["Pressao_arterial_sistolica"] < 60) |
4           (dados["Pressao_arterial_diastolica"] > 140
5           (dados["Pressao_arterial_diastolica"] < 30) |
6           (dados["IMC"] > 100) |
7           (dados["IMC"] < 5))
8 # Aplicação do filtro ao DataFrame para remover as linhas que
   não atendem as condições
9 dados = dados[~filtro]
```

Após o processo rigoroso de limpeza e transformação dos dados, foi possível realizar a exclusão de registros de baixa qualidade. O conjunto de dados que continha inicialmente 70.000 registros foi reduzido para 68.510 registros, resultando em um conjunto de dados refinado e de maior confiabilidade como mostra a Tabela 5. Essa abordagem meticulosa assegurou a remoção de dados inconsistentes ou incorretos, contribuindo para a melhoria da qualidade dos dados analisados. Com esse conjunto de dados depurado, as análises subsequentes podem ser conduzidas com maior precisão e confiabilidade, fornecendo *insights* mais robustos e embasados para tomadas de decisão informadas.

**Tabela 5** – Resumo estatístico dos dados após a limpeza e transformação.

|               | IMC       | Pressão arterial<br>sistólica | Pressão arterial<br>diastólica |
|---------------|-----------|-------------------------------|--------------------------------|
| Contagem      | 68510     | 68510                         | 68510                          |
| Media         | 27,383379 | 126,585243                    | 81,351584                      |
| Desvio Padrão | 5,043653  | 16,734730                     | 9,624099                       |
| Mínimo        | 10,726644 | 60,000000                     | 30,000000                      |
| Máximo        | 50,000000 | 240,000000                    | 170,000000                     |

## 4 DIVISÃO DOS DADOS

A divisão dos dados em conjuntos de treinamento e teste é uma prática comum em aprendizado de máquina e mineração de dados (HASTIE; TIBSHIRANI; FRIEDMAN, 2009; KOHAVI, 1995; BISHOP, 2006). O principal motivo dessa divisão é avaliar o desempenho do modelo em dados não vistos anteriormente e verificar sua capacidade de generalização (KIM, 2017).

Ao treinar um modelo de aprendizado de máquina, utiliza-se os dados disponíveis para aprender os padrões e relações entre as variáveis. No entanto, o objetivo final é que o modelo seja capaz de fazer previsões precisas em novos dados, ou seja, em dados que não foram usados durante o treinamento.

A divisão dos dados em conjuntos de treinamento e teste permite simular essa situação de prever novos dados. O conjunto de treinamento é usado para ajustar os parâmetros do modelo e encontrar as melhores configurações. O conjunto de teste, por sua vez, é usado para avaliar o desempenho do modelo, medindo sua capacidade de generalizar o conhecimento aprendido durante o treinamento (HASTIE; TIBSHIRANI; FRIEDMAN, 2009; KOHAVI, 1995; BISHOP, 2006).

A divisão dos dados de treino em 20% para teste e 80% para treino é uma prática comum em ciência de dados e *machine learning*. Essa divisão é conhecida como "validação cruzada" ou "separação treino-teste". A justificativa para essa proporção está relacionada a um equilíbrio entre ter dados suficientes para treinar o modelo de forma adequada e ter dados suficientes para avaliar o desempenho do modelo.

Ao separar 80% dos dados para treinamento, temos uma quantidade significativa de dados para que o modelo possa aprender e ajustar seus parâmetros. Isso ajuda a evitar problemas de subajuste, nos quais o modelo não é capaz de capturar os padrões complexos nos dados.

Os 20% restantes são reservados para teste, permitindo avaliar o desempenho do modelo em dados não vistos anteriormente. Essa parcela de dados não utilizada no treinamento é essencial para verificar se o modelo generaliza bem e é capaz de fazer previsões precisas em novas amostras.

Essa divisão foi realizada através do código abaixo:

```
1 from sklearn.model_selection import train_test_split
2
3 # A variável alvo (Doenca_cardiovascular) é separada das demais
   características
4 previsor = np.array(dados["Doenca_cardiovascular"])
5 caracteristicas = np.array(dados.drop("Doenca_cardiovascular",
   axis=1))
```

```

6 # A proporção para a divisão dos dados de teste é definida
7 test_size = 0.2 # 20% dos dados serão usados para teste
8 # A semente (random_state) é definida para garantir
   reprodutibilidade
9 random_state = 42 # Valor fixo para obter a mesma divisão dos
   dados toda vez que o código for executado
10 # Os dados são divididos em conjuntos de treinamento e teste
11 x_treino, x_teste, y_treino, y_teste =
   train_test_split(caracteristicas, previsor,
   test_size=test_size, random_state=random_state)

```

Nesse código, a função *train\_test\_split* do módulo *sklearn.model\_selection* é utilizada para dividir os dados em conjuntos de treinamento e teste. A variável *previsor* contém a coluna "Doença\_cardiovascular", que representa a variável alvo que será prevista. Já a variável *caracteristicas* contém as demais colunas, que são as características usadas para fazer as previsões.

O parâmetro *test\_size* define a proporção dos dados que serão utilizados para teste. Neste caso, foi definido como 0,2, o que significa que 20% dos dados serão separados para teste, enquanto os outros 80% serão usados para treinamento, essa é uma escolha comum quando tem-se um número suficientemente grande de amostras de dados.

O parâmetro *random\_state* é usado para garantir que a divisão dos dados seja a mesma em cada execução do código. Dessa forma, é possível obter resultados consistentes e reproduzíveis.

Por fim, os dados são divididos em quatro conjuntos: *x\_treino* e *y\_treino* contêm os dados de treinamento das características e da variável alvo, respectivamente, e *x\_teste* e *y\_teste* contêm os dados de teste das características e da variável alvo, respectivamente.

## 5 REGRESSÃO LOGÍSTICA

A regressão logística é um algoritmo amplamente utilizado para problemas de classificação binária, onde o objetivo é prever a probabilidade de um evento ocorrer. É particularmente adequado quando as variáveis independentes são lineares ou têm uma relação logística com a variável dependente (HASTIE; TIBSHIRANI; FRIEDMAN, 2009).

A biblioteca *scikit-learn* é uma ferramenta amplamente utilizada para aprendizado de máquina em *python*, e a classe *LogisticRegression* é uma implementação eficiente do algoritmo de regressão logística (PEDREGOSA et al., 2011). Ela fornece uma interface simples para treinar e usar modelos de regressão logística em uma variedade de problemas. Ao utilizar a biblioteca *scikit-learn* e a classe *LogisticRegression*, os usuários se beneficiam de várias vantagens:

- Implementação eficiente: A biblioteca *scikit-learn* é conhecida por sua implementação otimizada de algoritmos de aprendizado de máquina, incluindo a regressão logística. Ela foi desenvolvida levando em consideração o desempenho e a escalabilidade, o que é importante para conjuntos de dados grandes (PEDREGOSA et al., 2011).
- Facilidade de uso: A interface fornecida pela *scikit-learn* é intuitiva e amigável, permitindo que os usuários treinem e usem modelos de regressão logística com facilidade.

Em vista disso, o código abaixo foi desenvolvido para treinar um modelo de regressão logística:

```
1 from sklearn.linear_model import LogisticRegression
2
3 # Cria uma instância da classe LogisticRegression com os
  parâmetros especificados
4 funcao_reg_logis = LogisticRegression(penalty='l2', C=1.0,
  solver='sag', max_iter=1000)
5 # Treina o modelo utilizando os dados de treino
6 funcao_reg_logis.fit(x_treino, y_treino)
```

A escolha dos parâmetros também desempenha um papel fundamental no desempenho e na eficácia do modelo de regressão logística. Aqui está a justificativa para os parâmetros escolhidos:

- *penalty='l2'*: A regularização "l2"(ridge) é a escolha padrão, pois é eficaz para evitar *overfitting*, mantendo os coeficientes das variáveis próximos de zero. Isso ajuda a controlar a complexidade do modelo e a melhorar a generalização.

- $C=1.0$ : O parâmetro  $C$  controla a força da regularização. Um valor de 1.0 é um bom ponto de partida, pois indica uma regularização moderada.
- $solver='sag'$ : O solver "sag" é uma opção eficiente para grandes conjuntos de dados. Ele usa uma abordagem de descida de gradiente estocástica média e é mais rápido do que outros *solvers*, como "newton-cg" ou "lbfgs", para conjuntos de dados grandes.
- $max\_iter=1000$ : O número máximo de iterações é definido como 1000 para garantir que o algoritmo tenha tempo suficiente para convergir.

Com o modelo treinado, é possível fazer previsões em novos dados para realizar a classificação. Para isso, utiliza-se o modelo *funcao\_reg\_logis* treinado anteriormente para fazer previsões nos dados de teste *x\_teste*. As previsões são armazenadas na variável *previsoes\_log*, como mostra o código abaixo:

```
1 # Fazendo previsões com o modelo final
2 previsoes_log = funcao_reg_logis.predict(x_teste)
```

## 5.1 AVALIAÇÃO DO MODELO

Uma vez que as previsões foram realizadas, é essencial avaliar o desempenho do modelo utilizando métricas apropriadas. Uma métrica amplamente utilizada para avaliar modelos de classificação é a matriz de confusão. A biblioteca *sklearn.metrics* fornece a função *confusion\_matrix* que permite calcular a matriz de confusão, como mostra o código abaixo:

```
1 # Calculando a matriz de confusão comparando os rótulos
   verdadeiros (y_teste) com as previsões (previsoes_log)
2 matriz_confusao_log = confusion_matrix(y_teste, previsoes_log)
```

Como resultado tem-se a Tabela 6, que mostra a matriz confusão resultante da regressão logística. Recomenda-se ao leitor voltar a Tabela 1 para melhor interpretar a matriz confusão mostrada na Tabela 6.

**Tabela 6** – Matriz de confusão regressão logística.

|          | Positivo | Negativo |
|----------|----------|----------|
| Positivo | 5456     | 1471     |
| Negativo | 2278     | 4497     |

A partir da matriz de confusão, foram calculados as métricas acurácia, precisão, *recall* e *F1-score*, que são importantes para avaliar o desempenho do modelo. Os resultados dessas métricas são apresentados na Tabela 7.

**Tabela 7** – Estatística do modelo regressão logística.

| Modelo              | Acurácia | Precisão | <i>Recall</i> | <i>F1-score</i> |
|---------------------|----------|----------|---------------|-----------------|
| Regressão Logística | 0,719    | 0,787    | 0,705         | 0,744           |

Ao analisar as métricas do modelo de regressão logística para a predição de doenças cardiovasculares, pode-se obter *insights* sobre o desempenho do modelo:

- A acurácia do modelo é de 0,719, o que indica que o modelo classificou corretamente 71,9% dos casos. Essa é uma métrica geral que reflete a taxa de acertos do modelo em todas as classes. No entanto, é importante considerar que a acurácia pode ser afetada por desequilíbrios de classe e pode não fornecer uma imagem completa do desempenho do modelo.
- A precisão do modelo é de 0,787, o que significa que o modelo classificou corretamente 78,7% dos casos positivos. Essa métrica é relevante quando o foco está na minimização de falsos positivos, ou seja, quando é importante evitar a classificação incorreta de casos negativos como positivos.
- O *recall* do modelo é de 0,705, o que indica que o modelo identificou corretamente 70,5% dos casos positivos em relação ao total de casos positivos reais. Essa métrica é importante quando o objetivo é minimizar falsos negativos, ou seja, evitar a classificação incorreta de casos positivos como negativos.
- O *F1-score* é de 0,744, uma métrica que combina precisão e *recall* em um único valor, ele é útil quando se busca um equilíbrio entre a precisão e o *recall*, sendo uma medida adequada quando há desequilíbrio entre as classes ou quando ambas as classes são igualmente importantes.

Em conclusão, o desempenho do modelo de regressão logística foi considerado satisfatório por apresentar uma acurácia de 0,719 e um *F1-score* de 0,744. Esses valores indicam que o modelo foi capaz de classificar corretamente a grande maioria dos casos e alcançar um equilíbrio entre a precisão e o *recall*. Uma acurácia de 0,719 significa que o modelo acertou aproximadamente 71,9% das predições, o que é considerado um desempenho razoável. Além disso, o *F1-score* de 0,744, que combina a precisão e o *recall*, indica que o modelo foi capaz de encontrar um bom equilíbrio entre a identificação correta dos casos positivos e a minimização de falsos positivos e falsos negativos.

## 6 *K-NEAREST NEIGHBORS - K-NN*

A classificação k-vizinhos mais próximos *K-Nearest Neighbors - k-NN* é um algoritmo de aprendizado de máquina supervisionado que busca classificar um ponto de dados com base na sua proximidade com os vizinhos mais próximos no espaço de características (ALPAYDIN, 1992). O *k-NN* é amplamente utilizado em problemas de classificação devido à sua simplicidade e eficácia.

No contexto do algoritmo de *k-NN*, a biblioteca *scikit-learn* fornece a classe *KNeighborsClassifier* (PEDREGOSA et al., 2011), que implementa o algoritmo *k-NN* e permite ajustar o modelo aos dados de treinamento e realizar previsões em novos dados. O *KNeighborsClassifier* permite controlar vários parâmetros, como o número de vizinhos considerados (*n\_neighbors*), a métrica de distância utilizada (*metric*), entre outros.

Além disso, a biblioteca *scikit-learn* também fornece a classe *GridSearchCV* do módulo *model\_selection* (PEDREGOSA et al., 2011), que é uma ferramenta útil para otimizar os hiperparâmetros do modelo. O *GridSearchCV* realiza uma busca exaustiva por combinações de valores dos hiperparâmetros especificados, avaliando o desempenho do modelo para cada combinação por meio de validação cruzada. Dessa forma, é possível encontrar os melhores hiperparâmetros para o modelo de forma automatizada (HASTIE; TIBSHIRANI; FRIEDMAN, 2009).

Esses métodos, *KNeighborsClassifier* e *GridSearchCV*, são importados da biblioteca *scikit-learn* e são fundamentais para construir e ajustar o modelo *k-NN* de maneira eficiente. Eles são baseados em técnicas e algoritmos bem estabelecidos, amplamente documentados em literatura especializada (ALPAYDIN, 1992; HASTIE; TIBSHIRANI; FRIEDMAN, 2009).

Em vista disso, o código abaixo foi desenvolvido para treinar um modelo de *k-NN*:

```

1 from sklearn.neighbors import KNeighborsClassifier
2 from sklearn.model_selection import GridSearchCV
3
4 # Definindo os valores para os hiperparâmetros
5 param_grid = {
6     'n_neighbors': [3, 5, 7, 9, 11], # Valores para o número de
        vizinhos
7     'weights': ['uniform', 'distance'], # Opções para o peso
        dos vizinhos
8     'metric': ['euclidean', 'manhattan', 'minkowski'], #
        Métricas de distância
9     'p': [1, 2] # Valores para o parâmetro p da métrica de
        Minkowski
10 }
11 # Criando o objeto do modelo de k-NN

```



```

12 modelo_k-NN = KNeighborsClassifier()
13 # Realizando a busca pelos melhores hiperparâmetros usando
    validação cruzada
14 grid_search = GridSearchCV(modelo_k-NN, param_grid, cv=5)
15 # Treinando o modelo com os dados de treinamento
16 grid_search.fit(x_treino, y_treino)
17 # Obtendo os melhores hiperparâmetros encontrados
18 melhores_hiperparametros = grid_search.best_params_
19 # Criando o modelo final com os melhores hiperparâmetros
20 melhor_modelo_k-NN =
    KNeighborsClassifier(**melhores_hiperparametros)
21 # Treinando o modelo final com todos os dados de treinamento
22 melhor_modelo_k-NN.fit(x_treino, y_treino)

```

A escolha dos parâmetros também desempenha um papel fundamental no desempenho e na eficácia do modelo  $k$ -NN. Aqui está a justificativa para os parâmetros escolhidos:

- *n\_neighbors*: Foram considerados os valores [3, 5, 7, 9, 11] para o número de vizinhos. Essa escolha permite testar diferentes quantidades de vizinhos para a classificação. Valores menores podem resultar em modelos mais sensíveis ao ruído, enquanto valores maiores podem levar a uma perda de detalhes nas fronteiras de decisão.
- *weights*: Foram consideradas as opções '*uniform*' e '*distance*' para o peso dos vizinhos. A opção '*uniform*' atribui o mesmo peso para todos os vizinhos, enquanto '*distance*' atribui pesos inversamente proporcionais à distância. Essa escolha permite testar diferentes formas de considerar a influência dos vizinhos no processo de classificação.
- *metric*: Foram consideradas as métricas de distância '*euclidean*', '*manhattan*' e '*minkowski*'. A métrica '*euclidean*' corresponde à distância euclidiana, '*manhattan*' corresponde à distância de Manhattan e '*minkowski*' é uma métrica mais geral que inclui a distância euclidiana e a distância de Manhattan como casos especiais. Essa escolha permite testar diferentes medidas de distância, levando em consideração a geometria dos dados e a sensibilidade a diferentes distribuições de atributos.
- *p*: Foram considerados os valores [1, 2] para o parâmetro  $p$  da métrica de *Minkowski*. O valor 1 corresponde à distância de *Manhattan* e o valor 2 corresponde à distância *euclidiana*. Essa escolha permite testar diferentes graus de influência das diferentes dimensões dos atributos na distância calculada.

Com o modelo treinado, é possível fazer previsões em novos dados para realizar a classificação. Para isso, utiliza-se o modelo *melhor\_modelo\_k-NN* treinado anteriormente

para fazer previsões nos dados de teste  $x_{teste}$ . As previsões são armazenadas na variável  $previsoes\_k\text{-}NN$ , como mostra o código abaixo:

```
1 # Fazendo previsões com o modelo final
2 previsoes_k-NN = melhor_modelo_k-NN.predict(x_teste)
```

## 6.1 AVALIAÇÃO DO MODELO

Uma vez que as previsões foram realizadas, é essencial avaliar o desempenho do modelo utilizando métricas apropriadas. Uma métrica amplamente utilizada para avaliar modelos de classificação é a matriz de confusão. A biblioteca *sklearn.metrics* fornece a função *confusion\_matrix* que permite calcular a matriz de confusão, como mostra o código abaixo:

```
1 # Calculando a matriz de confusão comparando os rótulos
   verdadeiros (y_teste) com as previsões (previsoes_k-NN)
2 matriz_confusao = confusion_matrix(y_teste, previsoes_k-NN)
```

Como resultado tem-se a Tabela 8, que mostra a matriz confusão resultante do  $k\text{-}NN$ . Recomenda-se ao leitor voltar a Tabela 1 para melhor interpretar a matriz confusão mostrada na Tabela 8.

**Tabela 8** – Matriz de confusão  $k\text{-}NN$ .

|          | Positivo | Negativo |
|----------|----------|----------|
| Positivo | 5234     | 1693     |
| Negativo | 2257     | 4518     |

A partir da matriz de confusão, foram calculadas as métricas acurácia, precisão, *recall* e *F1-score*, que são importantes para avaliar o desempenho do modelo. Os resultados dessas métricas são apresentados na Tabela 9.

**Tabela 9** – Estatística do modelo  $k\text{-}NN$ .

| Modelo        | Acurácia | Precisão | <i>Recall</i> | <i>F1-score</i> |
|---------------|----------|----------|---------------|-----------------|
| $k\text{-}NN$ | 0,711    | 0,699    | 0,756         | 0,726           |

Ao analisar as métricas do modelo  $k\text{-}NN$  para a predição de doenças cardiovasculares, pode-se obter *insights* sobre o desempenho do modelo:

- A acurácia do modelo é de aproximadamente 71,1%, o que indica que o modelo classificou corretamente 71,1% dos casos. Isso significa que o modelo tem uma taxa de acertos razoável na classificação das amostras. No entanto, é importante considerar que a acurácia pode ser afetada por desequilíbrios de classe e pode não fornecer uma imagem completa do desempenho do modelo.
- A precisão do modelo é de aproximadamente 69,9%. Isso significa que o modelo classificou corretamente 69,9% dos casos positivos. A precisão é uma métrica relevante quando é importante evitar a classificação incorreta de casos negativos como positivos. No contexto de doenças cardiovasculares, uma alta precisão indica que o modelo tem uma boa capacidade de identificar corretamente os casos positivos, reduzindo assim os falsos positivos.
- O *recall* do modelo é de aproximadamente 75,6%. Isso indica que o modelo identificou corretamente 75,6% dos casos positivos em relação ao total de casos positivos reais. O *recall* é importante quando o objetivo é minimizar falsos negativos, ou seja, evitar a classificação incorreta de casos positivos como negativos. No contexto de doenças cardiovasculares, um alto *recall* indica que o modelo tem uma boa capacidade de detectar corretamente os casos positivos, reduzindo assim os falsos negativos.
- *F1-score*: O *F1-score* do modelo é de aproximadamente 0,726. O *F1-score* é uma métrica que combina precisão e *recall* em um único valor. Ele é útil quando se busca um equilíbrio entre a precisão e o *recall*. No contexto de doenças cardiovasculares, um *F1-score* alto indica que o modelo tem um bom equilíbrio entre a capacidade de identificar corretamente os casos positivos e a capacidade de evitar classificações incorretas.

Em conclusão, o modelo *k-NN* apresentou um desempenho satisfatório na predição de doenças cardiovasculares com uma acurácia de aproximadamente 71,1% e um *F1-score* de aproximadamente 0,726. Essas métricas indicam que o modelo foi capaz de classificar corretamente a maioria dos casos e alcançar um equilíbrio entre a precisão e o *recall*. A acurácia de 71,1% significa que o modelo acertou cerca de 71,1% das predições, o que é considerado um desempenho razoável. Além disso, o *F1-score* de 0,726, que combina a precisão e o *recall*, indica que o modelo encontrou um bom equilíbrio entre a identificação correta dos casos positivos e a minimização de falsos positivos e falsos negativos.

## 7 *RANDOM FOREST*

O algoritmo *random forest* é amplamente utilizado no campo de aprendizado de máquina como uma técnica eficaz para classificação e regressão. Ele combina várias árvores de decisão em uma "floresta", onde as previsões individuais de cada árvore são combinadas para obter uma previsão final mais robusta (BREIMAN, 2001).

No contexto do modelo *random forest*, dois métodos importantes são utilizados: o *RandomForestClassifier* e o *GridSearchCV*, ambos disponíveis na biblioteca *scikit-learn* (sklearn). O *RandomForestClassifier* é uma classe que implementa o algoritmo de *random forest* especificamente para tarefas de classificação (LIAW; WIENER, 2002). Ele permite a criação e treinamento de um modelo de *random forest* com facilidade.

Por outro lado, o *GridSearchCV* é responsável por realizar uma busca exaustiva pelos melhores hiperparâmetros do modelo de *random forest*, utilizando a técnica de validação cruzada. Ele permite a especificação de uma grade de valores possíveis para os hiperparâmetros, como o número de árvores na floresta, a profundidade máxima das árvores, entre outros. Ao realizar a validação cruzada, o *GridSearchCV* avalia o desempenho do modelo em diferentes combinações de hiperparâmetros, encontrando assim a configuração ideal (CUTLER et al., 2007).

Ao utilizar esses métodos em conjunto, é possível criar um modelo de *random forest* otimizado para tarefas de classificação, levando em consideração os melhores hiperparâmetros encontrados durante a busca exaustiva. Isso contribui para obter resultados mais precisos e eficientes na classificação de dados.

Em vista disso, o código abaixo foi desenvolvido para treinar um modelo de *random forest*:

```

1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.model_selection import GridSearchCV
3
4 # Definindo os valores possíveis para os hiperparâmetros
5 param_grid = {
6     'n_estimators': [100, 200], # Número de árvores na floresta
7     'max_depth': [None, 5], # Profundidade máxima das árvores
8     'min_samples_split': [2, 5], # Número mínimo de amostras
9     'min_samples_leaf': [1, 2], # Número mínimo de amostras
10    'max_features': ['sqrt', None] # Número de features a serem
11    consideradas ao procurar a melhor divisão
12 }
13 # Criando o modelo de Random Forest

```

```

13 modelo_rf = RandomForestClassifier()
14 # Realizando a busca pelos melhores hiperparâmetros usando
    validação cruzada
15 grid_search = GridSearchCV(modelo_rf, param_grid, cv=5)
16 # Treinando o modelo com os dados de treinamento
17 grid_search.fit(x_treino, y_treino)
18 # Obtendo os melhores hiperparâmetros encontrados
19 melhores_hiperparametros = grid_search.best_params_
20 # Criando o modelo final com os melhores hiperparâmetros
21 melhor_modelo_rf =
    RandomForestClassifier(**melhores_hiperparametros)
22 # Treinando o modelo final com todos os dados de treinamento
23 melhor_modelo_rf.fit(x_treino, y_treino)

```

A escolha dos parâmetros desempenha um papel fundamental no desempenho e na eficácia do modelo *random forest*. Foram definidos diferentes valores para cada um dos hiperparâmetros do modelo (LIAW; WIENER, 2002).

O número de árvores na floresta (*n\_estimators*) foi variado entre 100 e 200. Essa escolha permite avaliar o impacto do número de árvores no desempenho do modelo.

A profundidade máxima das árvores (*max\_depth*) foi ajustada considerando os valores *None* e 5. Isso permite controlar a complexidade das árvores, evitando *overfitting* ou *underfitting*.

O número mínimo de amostras necessárias para dividir um nó (*min\_samples\_split*) foi explorado com os valores 2 e 5. Essa configuração afeta a forma como as árvores são construídas, influenciando a capacidade do modelo de capturar relações mais complexas nos dados.

O número mínimo de amostras necessárias em um nó folha (*min\_samples\_leaf*) foi ajustado com os valores 1 e 2. Essa escolha permite controlar a quantidade mínima de amostras em um nó para que ele seja considerado uma folha, evitando folhas com muito poucas amostras.

O número de *features* consideradas ao procurar a melhor divisão (*max\_features*) foi explorado com as opções 'sqrt' e *None*. Essa configuração controla o número máximo de *features* a serem consideradas em cada divisão e pode impactar a diversidade das árvores na floresta.

Em seguida, utilizando a classe *GridSearchCV*, foi realizado um processo de busca exaustiva pelos melhores hiperparâmetros do modelo de *random forest*, com validação cruzada com 5 dobras (CUTLER et al., 2007). Essa abordagem permite avaliar o desempenho do modelo em diferentes combinações de hiperparâmetros, selecionando aqueles que resultam em um melhor desempenho médio durante a validação cruzada.

Após a busca, os melhores hiperparâmetros encontrados foram utilizados para criar um novo modelo de *random forest*. Esse modelo final, obtido com os melhores hiperparâmetros, foi treinado utilizando todos os dados de treinamento. Essa etapa garante que o modelo final esteja otimizado com os melhores hiperparâmetros encontrados, proporcionando um desempenho aprimorado para a tarefa de classificação.

Com o modelo treinado, é possível fazer previsões em novos dados para realizar a classificação. Para isso, utiliza-se o modelo *melhor\_modelo\_rf* treinado anteriormente para fazer previsões nos dados de teste *x\_teste*. As previsões são armazenadas na variável *previsoes\_rf*, como mostra o código abaixo:

```
1 # Realizando as previsões com os dados de teste
2 previsoes_rf = melhor_modelo_rf.predict(x_teste)
```

## 7.1 AVALIAÇÃO DO MODELO

Uma vez que as previsões foram realizadas, é essencial avaliar o desempenho do modelo utilizando métricas apropriadas. Uma métrica amplamente utilizada para avaliar modelos de classificação é a matriz de confusão. A biblioteca *sklearn.metrics* fornece a função *confusion\_matrix* que permite calcular a matriz de confusão, como mostra o código abaixo:

```
1 # Calculando a matriz de confusão comparando os rótulos
   verdadeiros (y_teste) com as previsões (previsoes_rf)
2 matriz_confusao_rf = confusion_matrix(y_teste, previsoes_rf)
```

Como resultado tem-se a Tabela 10, que mostra a matriz confusão resultante do *random forest*. Recomenda-se ao leitor voltar a Tabela 1 para melhor interpretar a matriz confusão mostrada na Tabela 10.

**Tabela 10** – Matriz de Confusão *random forest*.

|          | Positivo | Negativo |
|----------|----------|----------|
| Positivo | 2051     | 423      |
| Negativo | 814      | 1621     |

A partir da matriz de confusão, foram calculadas as métricas acurácia, precisão, *recall* e *F1-score*, que são importantes para avaliar o desempenho do modelo. Os resultados dessas métricas são apresentados na Tabela 11.

**Tabela 11** – Estatística do modelo *random forest*.

| Modelo               | Acurácia | Precisão | <i>Recall</i> | <i>F1-score</i> |
|----------------------|----------|----------|---------------|-----------------|
| <i>random forest</i> | 0,7480   | 0,7930   | 0,6657        | 0,7238          |

Ao analisar as métricas do modelo *random forest* para a predição de doenças cardiovasculares, pode-se obter *insights* sobre o desempenho do modelo:

- A acurácia do modelo é de aproximadamente 74,80%, o que indica que o modelo classificou corretamente 74,80% dos casos. Isso significa que o modelo apresenta uma taxa de acertos satisfatória na classificação das amostras. No entanto, é importante considerar que a acurácia pode ser afetada por desequilíbrios de classe e pode não fornecer uma imagem completa do desempenho do modelo.
- A precisão do modelo é de aproximadamente 79,30%. Isso significa que o modelo classificou corretamente 79,30% dos casos positivos. A precisão é uma métrica relevante quando é importante evitar a classificação incorreta de casos negativos como positivos. No contexto de doenças cardiovasculares, uma alta precisão indica que o modelo possui uma boa capacidade de identificar corretamente os casos positivos, reduzindo assim os falsos positivos.
- A precisão do modelo é de aproximadamente 79,30%. Isso significa que o modelo classificou corretamente 79,30% dos casos positivos. A precisão é uma métrica relevante quando é importante evitar a classificação incorreta de casos negativos como positivos. No contexto de doenças cardiovasculares, uma alta precisão indica que o modelo possui uma boa capacidade de identificar corretamente os casos positivos, reduzindo assim os falsos positivos.
- O *recall* do modelo é de aproximadamente 66,57%. Isso indica que o modelo identificou corretamente 66,57% dos casos positivos em relação ao total de casos positivos reais. O *recall* é importante quando o objetivo é minimizar falsos negativos, ou seja, evitar a classificação incorreta de casos positivos como negativos. No contexto de doenças cardiovasculares, um alto *recall* indica que o modelo possui uma boa capacidade de detectar corretamente os casos positivos, reduzindo assim os falsos negativos.
- O *F1-score* do modelo é de aproximadamente 0,7238. O *F1-score* é uma métrica que combina precisão e *recall* em um único valor. Essa métrica é útil quando se busca um equilíbrio entre a precisão e o *recall*. No contexto de doenças cardiovasculares, um *F1-score* alto indica que o modelo possui um bom equilíbrio entre a capacidade

de identificar corretamente os casos positivos e a capacidade de evitar classificações incorretas.

Em conclusão, o modelo *random forest* apresentou um desempenho satisfatório na predição de doenças cardiovasculares, com uma acurácia de aproximadamente 74,80% e um *F1-score* de aproximadamente 0,7238. Essas métricas indicam que o modelo foi capaz de classificar corretamente a maioria dos casos e alcançar um equilíbrio entre a precisão e o *recall*. A acurácia de 74,80% significa que o modelo acertou cerca de 74,80% das predições, o que é considerado um desempenho satisfatório. Além disso, o *F1-score* de 0,7238, que combina a precisão e o *recall*, indica que o modelo encontrou um bom equilíbrio entre a identificação correta dos casos positivos e a minimização de falsos positivos e falsos negativos.



## 8 RESULTADOS

Nesta análise, foram considerados três modelos diferentes para a predição de doenças cardiovasculares: regressão logística,  $k$ -NN e *random forest*. A Tabela 12 apresenta as estatísticas de desempenho desses modelos, incluindo acurácia, precisão, *recall* e *F1-score*. Essas métricas fornecem *insights* sobre a capacidade dos modelos em classificar corretamente os casos positivos e negativos, bem como o equilíbrio entre essas classificações.

**Tabela 12** – Estatística dos 3 modelos.

| Modelo               | Acurácia | Precisão | <i>Recall</i> | <i>F1-score</i> |
|----------------------|----------|----------|---------------|-----------------|
| Regressão Logística  | 0,719    | 0,787    | 0,705         | 0,744           |
| $k$ -NN              | 0,711    | 0,699    | 0,756         | 0,726           |
| <i>Random Forest</i> | 0,7480   | 0,7930   | 0,6657        | 0,7238          |

Ao analisar os resultados da tabela, pode-se observar que todos os três modelos apresentaram um desempenho geral satisfatório na predição de doenças cardiovasculares. No entanto, algumas diferenças podem ser destacadas.

A regressão logística obteve uma acurácia de 71,9%, o que significa que classificou corretamente cerca de 71,9% dos casos. A precisão foi de 78,7%, indicando que o modelo classificou corretamente 78,7% dos casos positivos. O *recall* foi de 70,5%, o que significa que o modelo identificou corretamente 70,5% dos casos positivos em relação ao total de casos positivos reais. O *F1-score* foi de 74,4%, evidenciando um equilíbrio entre a precisão e o *recall*.

O modelo  $k$ -NN apresentou uma acurácia de 71,1%, o que indica uma taxa de acertos razoável na classificação das amostras. A precisão foi de 69,9%, evidenciando que classificou corretamente 69,9% dos casos positivos. O *recall* foi de 75,6%, indicando uma boa capacidade de detectar corretamente os casos positivos. O *F1-score* foi de 72,6%, mostrando um bom equilíbrio entre a identificação correta dos casos positivos e a minimização de classificações incorretas.

O modelo *random forest* obteve a maior acurácia entre os três modelos, com 74,8%. A precisão foi de 79,3%, indicando uma boa capacidade de identificar corretamente os casos positivos. O *recall* foi de 66,6%, o que significa que o modelo identificou corretamente 66,6% dos casos positivos em relação ao total de casos positivos reais. O *F1-score* foi de 72,4%, mostrando um equilíbrio entre a precisão e o *recall*.

Em termos gerais, os três modelos apresentaram resultados próximos em relação à acurácia e ao *F1-score*. No entanto, a Regressão Logística e o *Random Forest* obtiveram uma precisão ligeiramente maior em comparação com o  $k$ -NN. Por outro lado, o  $k$ -

*NN* apresentou o maior *recall*, indicando uma melhor capacidade de detecção dos casos positivos.

Com base nas estatísticas fornecidas pela tabela, conclui-se que todos os três modelos de regressão logística, *k-NN* e *random forest*, apresentaram um desempenho satisfatório na predição de doenças cardiovasculares. Cada modelo possui suas vantagens e características específicas, como a capacidade de evitar classificações incorretas, identificar corretamente os casos positivos ou encontrar um equilíbrio entre a precisão e o *recall*.

## 9 CONCLUSÃO

Em conclusão, o presente estudo teve como objetivo a aplicação de diferentes algoritmos de aprendizado de máquina para a tarefa de predição de doenças cardiovasculares. Três modelos foram avaliados: regressão logística, *k-NN* e *random forest*. Ao longo do trabalho, foram exploradas as diversas bibliotecas e ferramentas disponíveis em *python* para o desenvolvimento de modelos de aprendizado de máquina. O *python* destaca-se como uma linguagem de programação popular e amplamente utilizada no campo da ciência de dados, devido à sua facilidade de uso, ampla gama de bibliotecas especializadas e uma comunidade ativa.

Através da biblioteca *scikit-learn*, foram implementados e avaliados os modelos de forma eficiente, aproveitando as funções pré-existentes para o treinamento, teste e avaliação dos algoritmos. Isso permitiu uma abordagem mais ágil e precisa na construção e análise dos modelos de aprendizado de máquina utilizados neste estudo.

Com base nos resultados obtidos, conclui-se que todos os três modelos de regressão logística, *k-NN* e *random forest*, apresentaram um desempenho satisfatório na predição de doenças cardiovasculares. Cada modelo demonstrou suas vantagens e características específicas, como a capacidade de evitar classificações incorretas, identificar corretamente os casos positivos ou encontrar um equilíbrio entre precisão e *recall*. Esses *insights* contribuem para uma melhor compreensão da eficácia desses modelos na área de previsão de doenças cardiovasculares e destacam a importância de escolher abordagens adequadas de aprendizado de máquina para problemas de saúde.

Como trabalho futuro, propõe-se o desenvolvimento de um software público que permita aos profissionais da saúde adicionar novos dados ao banco e visualizar previsões para obter *insights* relevantes. A disponibilização desse software público teria o potencial de melhorar o monitoramento e a predição de doenças cardiovasculares, permitindo que os profissionais da saúde tenham acesso a informações atualizadas e precisas para auxiliar em suas práticas clínicas. Além disso, a adição contínua de novos dados e integração de outras bases de dados ao banco atual contribuiria para a melhoria e refinamento dos modelos de predição, tornando-os ainda mais eficazes ao longo do tempo.

## Referências

- AGGARWAL, Charu C. *Deep Learning*. [S. l.]: Springer, 2018.
- ALPAYDIN, Ethem. *Introduction to Machine Learning*. Cambridge, MA: MIT Press, 1992.
- AMERICAN HEART ASSOCIATION. *Causes of Heart Disease*. [S. l.: s. n.], 2021. <https://www.heart.org/en/health-topics/congenital-heart-defects/the-impact-of-congenital-heart-defects/causes-of-heart-defects>. Acesso em 27 de abril de 2023.
- AMERICAN HEART ASSOCIATION. *Types of Heart Disease*. [S. l.: s. n.], 2021. <https://www.heart.org/en/health-topics/congenital-heart-defects/the-impact-of-congenital-heart-defects/types-of-heart-defects>. Acesso em 27 de abril de 2023.
- ASSOCIATION, American Heart. *Understanding Blood Pressure Readings*. [S. l.: s. n.], 2023. <https://www.heart.org/en/health-topics/high-blood-pressure/understanding-blood-pressure-readings>. Acesso em: 15 de abril de 2023.
- BAROCAS, Solon; SELBST, Andrew D. *Fairness and Machine Learning*. [S. l.]: MIT Press, 2020.
- BISHOP, Christopher M. *Pattern recognition and machine learning*. New York, NY: Springer, 2006. DOI: 10.1007/978-0-387-84858-7.
- BREIMAN, Leo. Random forests. *Machine learning*, Springer, v. 45, n. 1, p. 5–32, 2001. DOI: 10.1023/A:1010933404324.
- CUTLER, D. Richard et al. Random forests for classification in ecology. *Ecology*, Ecological Society of America, v. 88, n. 11, p. 2783–2792, 2007.
- DOMINGOS, P. *The Master Algorithm: How the Quest for the Ultimate Learning Machine Will Remake Our World*. [S. l.]: Basic Books, 2018.
- EUROPEAN SOCIETY OF CARDIOLOGY. *European Society of Cardiology: Blood Pressure Guidelines*. [S. l.], 2023. Acesso em: 15 de abril de 2023. Disponível em: <<https://www.escardio.org/Guidelines/Clinical-Practice-Guidelines/Hypertension-Management>>.
- GALAR, Mikel et al. A review on ensembles for the class imbalance problem: Bagging-, boosting-, and hybrid-based approaches. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, IEEE, v. 42, n. 4, p. 463–484, 2012. DOI: 10.1109/TSMCC.2011.2161285. Disponível em: <<https://doi.org/10.1109/TSMCC.2011.2161285>>.
- GERON, Aurelien. *Hands-On Machine Learning with Scikit-Learn and TensorFlow*. [S. l.]: O'Reilly Media, 2017.

- GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. *Deep Learning*. [S. l.]: MIT Press, 2016.
- HASTIE, Trevor; TIBSHIRANI, Robert; FRIEDMAN, Jerome. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. [S. l.]: Springer, 2009.
- JAPKOWICZ, Nathalie; SHAH, Mohak. *Evaluating learning algorithms: A classification perspective*. [S. l.]: Cambridge University Press, 2011. DOI: 10.1017/CB09780511921809.
- KAGGLE. *Kaggle*. Acesso em 2023. Disponível em: <<https://www.kaggle.com>>.
- KIM, Jinho. Comparative study on machine learning techniques in data classification. *Journal of Information Processing Systems*, KIPS (Korea Information Processing Society), Seoul, South Korea, v. 13, n. 1, p. 106–116, 2017. DOI: 10.3745/JIPS.04.0030.
- KOHAVI, Ron. A study of cross-validation and bootstrap for accuracy estimation and model selection. In: MORGAN KAUFMANN, 2-3. INTERNATIONAL Joint Conference on Artificial Intelligence. San Francisco, CA: [s. n.], 1995. v. 14, p. 1137–1145. DOI: 10.1007/978-3-642-96018-5\_3.
- KRITTANAWONG, C.; JOHNSON, K. W.; ROSENSON, R. S. et al. Deep learning for cardiovascular medicine: A practical primer. *European Heart Journal*, v. 40, n. 25, p. 2058–2073, 2019. DOI: 10.1093/eurheartj/ehz525.
- LIAW, Andy; WIENER, Matthew. Classification and regression by randomForest. *R News*, v. 2, n. 3, p. 18–22, 2002. DOI: 10.1007/s00180-014-0503-2.
- MITCHELL, Tom M. *Machine Learning*. [S. l.]: McGraw-Hill, 1997.
- MURPHY, Kevin P. *Machine Learning: A Probabilistic Perspective*. [S. l.]: The MIT Press, 2012.
- MUSA, A. M.; ULLAH, I.; BISWAS, M. et al. Cardiovascular risk prediction models using machine learning techniques: A systematic review. *PLoS ONE*, v. 15, n. 9, e0239179, 2020. DOI: 10.1371/journal.pone.0239179.
- ORGANIZAÇÃO MUNDIAL DA SAÚDE. *Doenças cardiovasculares*. [S. l.], 2023. Acessado em: 16 de abril de 2023. Disponível em: <[https://www.who.int/health-topics/cardiovascular-diseases/#tab=tab\\_1](https://www.who.int/health-topics/cardiovascular-diseases/#tab=tab_1)>.
- PEDREGOSA, F. et al. *Scikit-learn: Machine Learning in Python*. 2011. Disponível em: <<https://scikit-learn.org>>.
- POWERS, David M. Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation. *Journal of Machine Learning Technologies*, v. 2, n. 1, p. 37–63, 2021. DOI: 10.2166/2.1.006.

- RUSSELL, S. J.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. [S. l.]: Pearson, 2009.
- SUTTON, Richard S.; BARTO, Andrew G. *Reinforcement Learning: An Introduction*. [S. l.]: The MIT Press, 2018.
- UNSUPERVISED Learning. 1st. New York, NY: Springer, 2009.
- WORLD HEALTH ORGANIZATION. *Cardiovascular Diseases (CVDs)*. [S. l.: s. n.], 2021.  
[https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-\(cvds\)](https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds)).
- WORLD HEALTH ORGANIZATION. *Obesity and overweight*. Acessado em: 01 de abril de 2023. 2022. Disponível em: <<https://www.who.int/news-room/fact-sheets/detail/obesity-and-overweight>>.
- ZHANG, Xiao-Feng; OU-YANG, Lun. Machine learning and artificial intelligence in bioinformatics: Beyond supervised learning. *Bioinformatics*, Oxford University Press, v. 34, n. 19, p. 3267–3269, 2018. DOI: 10.1093/bioinformatics/bty304. Disponível em: <<https://doi.org/10.1093/bioinformatics/bty304>>.
- ZHANG, Y.; LIU, Q. *Interpretable Machine Learning: Methods and Applications*. [S. l.]: Springer, 2021.

## ANEXO A - AMOSTRA COMPLETA DO CONJUNTO DE DADOS ORIGINAL

Link para acessar o conjunto de dados no *Kaggle* <https://www.kaggle.com/datasets/sulianova/cardiovascular-disease-dataset>.

**Tabela 13** – Amostra completa dos registros do conjunto de dados original (Parte 1).

| Id    | Idade | Gênero | Altura | Peso  | Pressão arterial | Pressão arterial |
|-------|-------|--------|--------|-------|------------------|------------------|
|       |       |        |        |       | sistólica        | diastólica       |
| 0     | 18393 | 2      | 168    | 62,0  | 110              | 80               |
| 1     | 20228 | 1      | 156    | 85,0  | 140              | 90               |
| 2     | 18857 | 1      | 165    | 64,0  | 130              | 70               |
| 3     | 17623 | 2      | 169    | 82,0  | 150              | 100              |
| 4     | 17474 | 1      | 156    | 56,0  | 100              | 60               |
| 69995 | 19240 | 2      | 168    | 76,0  | 120              | 80               |
| 69996 | 22601 | 1      | 158    | 126,0 | 140              | 90               |
| 69997 | 19066 | 2      | 183    | 105,0 | 180              | 90               |
| 69998 | 22431 | 1      | 163    | 72,0  | 135              | 80               |
| 69999 | 20540 | 1      | 170    | 72,0  | 120              | 80               |

**Tabela 14** – Amostra completa dos registros do conjunto de dados original (Parte 2).

|   | Colesterol | Glicose | Fumante | Consumo de | Atividade | Doença         |
|---|------------|---------|---------|------------|-----------|----------------|
|   |            |         |         | álcool     | física    | cardiovascular |
| 1 | 1          | 0       | 0       | 0          | 1         | 0              |
| 3 | 1          | 0       | 0       | 0          | 1         | 1              |
| 3 | 1          | 0       | 0       | 0          | 0         | 1              |
| 1 | 1          | 0       | 0       | 0          | 1         | 1              |
| 1 | 1          | 0       | 0       | 0          | 0         | 0              |
| 1 | 1          | 1       | 0       | 0          | 1         | 0              |
| 2 | 2          | 0       | 0       | 0          | 1         | 1              |
| 3 | 1          | 0       | 1       | 0          | 0         | 1              |
| 1 | 2          | 0       | 0       | 0          | 0         | 1              |
| 2 | 1          | 0       | 0       | 0          | 1         | 0              |

## ANEXO B - CÓDIGO COMPLETO

Link para acessar o código do trabalho no *GitHub* [https://github.com/PedroAssisDev/Tecnicas\\_ML\\_Predicao\\_Doencas\\_Cardiovasculares](https://github.com/PedroAssisDev/Tecnicas_ML_Predicao_Doencas_Cardiovasculares).

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib as plt
4 import seaborn as sns
5 from datetime import timedelta
6 # Carrega o arquivo CSV em um DataFrame
7 dados = pd.read_csv("/content/cardio_train.csv", sep=";")
8 # Renomeia as colunas do DataFrame
9 dados.columns = ["Id", "Idade", "Genero", "Altura", "Peso",
10                  "Pressao_artestinal_sistolica", "Pressao_artestinal_diastolica",
11                  "Colesterol", "Glicose", "Fumante",
12                  "Consumo_de_alcool", "Atividade_fisica",
13                  "Doenca_cardiovascular"]
14 # Exibe informações sobre o DataFrame
15 print(dados.info())
16 from sklearn.metrics import accuracy_score, precision_score,
17     recall_score, f1_score
18 # Função para calculo dos parametros
19 def calculo_estatistico(confusion_matrix):
20     true_negative = confusion_matrix[0, 0]
21     false_positive = confusion_matrix[0, 1]
22     false_negative = confusion_matrix[1, 0]
23     true_positive = confusion_matrix[1, 1]
24     # Cálculo da Acurácia
25     accuracy = (true_negative + true_positive) /
26         confusion_matrix.sum()
27     # Cálculo da Precisão
28     precision = true_positive / (true_positive + false_positive)
29     # Cálculo do Recall
30     recall = true_positive / (true_positive + false_negative)
31     # Cálculo do F1-score
32     f1 = 2 * (precision * recall) / (precision + recall)
33     print("Acurácia:", accuracy)
34     print("Precisão:", precision)
35     print("Recall:", recall)
36     print("F1-score:", f1)
37 ##### Tratamento de Dados #####

```



```

33 # Remover a coluna "id" do DataFrame
34 dados.drop("Id", axis=1, inplace=True)
35 # Remover linhas duplicadas do DataFrame
36 dados.drop_duplicates(inplace=True)
37 # Converter a coluna "Idade" de dias para anos
38 dados['Idade'] = dados['Idade'].apply(lambda x:
    timedelta(days=x) / timedelta(days=365.25))
39 # Calcular o Índice de Massa Corporal (IMC) e adicionar uma nova
    coluna "IMC" ao DataFrame
40 dados["IMC"] = dados["Peso"] / (dados["Altura"]/100)**2
41 # Mudar ordem das colunas
42 dados = dados[["Idade", "Genero", "Altura", "Peso", "IMC",
    "Pressao_artestinal_sistolica",
43                 "Pressao_artestinal_diastolica", "Colesterol",
    "Glicose", "Fumante", "Consumo_de_alcool",
    "Atividade_fisica"]
44 dados.head()
45 dados.describe()
46 # Criação do filtro para selecionar linhas com base em condições
    específicas
47 filtro = ((dados["Pressao_artestinal_sistolica"] > 220) |
48           (dados["Pressao_artestinal_sistolica"] < 60) |
49           (dados["Pressao_artestinal_diastolica"] > 140
50           (dados["Pressao_artestinal_diastolica"] < 30) |
51           (dados["IMC"] > 100) |
52           (dados["IMC"] < 5))
53 # Aplicação do filtro ao DataFrame para remover as linhas que
    não atendem as condições
54 dados = dados[~filtro]
55 # Exibição das primeiras linhas do DataFrame após a aplicação do
    filtro
56 dados.head()
57 dados.describe()
58 # Plot para a visualização
59 sns.set(font_scale = 1.3, rc = {'figure.figsize':(20,20)})
60 eixo = dados.hist(bins = 70)
61
62 ##### Modelo #####
63 from sklearn.model_selection import train_test_split
64 # A variável alvo (Doenca_cardiovascular) é separada das demais
    características
65 previsor = np.array(dados["Doenca_cardiovascular"])

```

```

66 caracteristicas = np.array(dados.drop("Doenca_cardiovascular",
    axis=1))
67 # A proporção para a divisão dos dados de teste é definida
68 test_size = 0.2 # 20% dos dados serão usados para teste
69 # A semente (random_state) é definida para garantir
    reprodutibilidade
70 random_state = 42 # Valor fixo para obter a mesma divisão dos
    dados toda vez que o código for executado
71 # Os dados são divididos em conjuntos de treinamento e teste
72 x_treino, x_teste, y_treino, y_teste =
    train_test_split(caracteristicas, previsor,
        test_size=test_size, random_state=random_state)
73 print(f'{len(caracteristicas)}\n{len(previsor)}')
74 print(f'x_treino {len(x_treino)}, x_teste {len(x_teste)}')
75 print(f'x_treino {len(x_treino)/len(caracteristicas)}, x_teste
    {len(x_teste)/len(caracteristicas)}')
76 ##### Regressão Logistica #####
77 # LogisticRegression
78 LogisticRegression(max_iter=1000, solver='sag')
79 from sklearn.linear_model import LogisticRegression
80 # Cria uma instância da classe LogisticRegression com os
    parâmetros especificados
81 funcao_reg_logis = LogisticRegression(penalty='l2', C=1.0,
    solver='sag', max_iter=1000)
82 # Treina o modelo utilizando os dados de treino
83 funcao_reg_logis.fit(x_treino, y_treino)
84 # Fazendo previsões com o modelo final
85 previsoes_log = funcao_reg_logis.predict(x_teste)
86 from sklearn.metrics import confusion_matrix
87 # Calculando a matriz de confusão comparando os rótulos
    verdadeiros (y_teste) com as previsões (previsoes_log)
88 matriz_confusao_log = confusion_matrix(y_teste, previsoes_log)
89 print(matriz_confusao_log)
90 calculo_estatistico(matriz_confusao_log)
91 from sklearn.metrics import classification_report
92 print(classification_report(y_teste, previsoes_log))
93 ##### KNN #####
94 from sklearn.neighbors import KNeighborsClassifier
95 from sklearn.model_selection import GridSearchCV
96 # Definindo os valores para os hiperparâmetros
97 param_grid = {

```

```

98     'n_neighbors': [3, 5, 7, 9, 11], # Valores para o número de
      vizinhos
99     'weights': ['uniform', 'distance'], # Opções para o peso
      dos vizinhos
100    'metric': ['euclidean', 'manhattan', 'minkowski'], #
      Métricas de distância
101    'p': [1, 2] # Valores para o parâmetro p da métrica de
      Minkowski
102 }
103 # Criando o objeto do modelo de KNN
104 modelo_knn = KNeighborsClassifier()
105 # Realizando a busca pelos melhores hiperparâmetros usando
      validação cruzada
106 grid_search = GridSearchCV(modelo_knn, param_grid, cv=5)
107 # Treinando o modelo com os dados de treinamento
108 grid_search.fit(x_treino, y_treino)
109 # Obtendo os melhores hiperparâmetros encontrados
110 melhores_hiperparametros = grid_search.best_params_
111 # Criando o modelo final com os melhores hiperparâmetros
112 melhor_modelo_knn =
      KNeighborsClassifier(**melhores_hiperparametros)
113 # Treinando o modelo final com todos os dados de treinamento
114 melhor_modelo_knn.fit(x_treino, y_treino)
115 # Fazendo previsões com o modelo final
116 previsoes_knn = melhor_modelo_knn.predict(x_teste)
117 # Calculando a matriz de confusão comparando os rótulos
      verdadeiros (y_teste) com as previsões (previsoes_knn)
118 matriz_confusao_knn = confusion_matrix(y_teste, previsoes_knn)
119 print(matriz_confusao_knn)
120 calculo_estatistico(matriz_confusao_knn)
121 from sklearn.metrics import classification_report
122 print(classification_report(y_teste, previsoes_knn))
123 ##### RANDOM FOREST #####
124 from sklearn.ensemble import RandomForestClassifier
125 from sklearn.model_selection import GridSearchCV
126 # Definindo os valores possíveis para os hiperparâmetros
127 param_grid = {
128     'n_estimators': [100, 200], # Número de árvores na floresta
129     'max_depth': [None, 5], # Profundidade máxima das árvores
130     'min_samples_split': [2, 5], # Número mínimo de amostras
      necessárias para dividir um nó

```

```

131     'min_samples_leaf': [1, 2], # Número mínimo de amostras
        necessárias em um nó folha
132     'max_features': ['sqrt', None] # Número de features a serem
        consideradas ao procurar a melhor divisão
133 }
134 # Criando o modelo de Random Forest
135 modelo_rf = RandomForestClassifier()
136 # Realizando a busca pelos melhores hiperparâmetros usando
        validação cruzada
137 grid_search = GridSearchCV(modelo_rf, param_grid, cv=5)
138 # Treinando o modelo com os dados de treinamento
139 grid_search.fit(x_treino, y_treino)
140 # Obtendo os melhores hiperparâmetros encontrados
141 melhores_hiperparametros = grid_search.best_params_
142 # Criando o modelo final com os melhores hiperparâmetros
143 melhor_modelo_rf =
        RandomForestClassifier(**melhores_hiperparametros)
144 # Treinando o modelo final com todos os dados de treinamento
145 melhor_modelo_rf.fit(x_treino, y_treino)
146 # Realizando as previsões com os dados de teste
147 previsoes_rf = melhor_modelo_rf.predict(x_teste)
148 # Calculando a matriz de confusão comparando os rótulos
        verdadeiros (y_teste) com as previsões (previsoes_rf)
149 matriz_confusao_rf = confusion_matrix(y_teste, previsoes_rf)
150 print(matriz_confusao_rf)
151 calculo_estatistico(matriz_confusao_rf)
152 from sklearn.metrics import classification_report
153 print(classification_report(y_teste, previsoes_rf))

```